Learning When to Adapt

Fine-Tuning with Less Forgetting

Ali Zindari

CISPA · Saarland University

Supervisor: Sebastian U. Stich

Three papers on fine-tuning

2026

Learning When to Adapt

Ali Zindari
Xiaowen Jiang
Rotem Mulayoff
Sebastian U. Stich

Under review [Paper]

DISeL · Main focus today

Three papers on fine-tuning

2026

Learning When to Adapt

Ali Zindari
Xiaowen Jiang
Rotem Mulayoff
Sebastian U. Stich

Under review [Paper]

DISeL · Main focus today

2026

LoRA vs. Full Fine-Tuning: A Theoretical Perspective

Ali Zindari*
Rotem Mulayoff*
Sebastian U. Stich

* Equal contribution

Under review [Paper]

When low rank helps

Three papers on fine-tuning

2026

Learning When to Adapt

Ali Zindari
Xiaowen Jiang
Rotem Mulayoff
Sebastian U. Stich

Under review [Paper]

DISeL · Main focus today

2026

LoRA vs. Full Fine-Tuning: A Theoretical Perspective

Ali Zindari*
Rotem Mulayoff*
Sebastian U. Stich

* Equal contribution

Under review [Paper]

When low rank helps

2026

Theory of Adaptation and Retention

Ali Zindari
Rotem Mulayoff
Bruno Loureiro
Sebastian U. Stich

Under review [Manuscript]

Limits of learning and retention

Pre-training: the first stage of LLM training

Large text collections



Books · websites · articles
Across many topics

Pre-training: the first stage of LLM training

Large text collections



Books · websites · articles
Across many topics

Train from scratch

High compute cost
Long training time

Pre-training: the first stage of LLM training

Large text collections

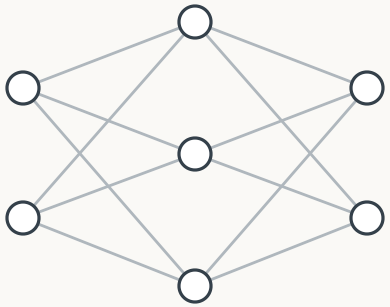


Books · websites · articles
Across many topics

Train from scratch

High compute cost
Long training time

Pre-trained model



Language understanding
General knowledge

Pre-training: the first stage of LLM training

Large text collections

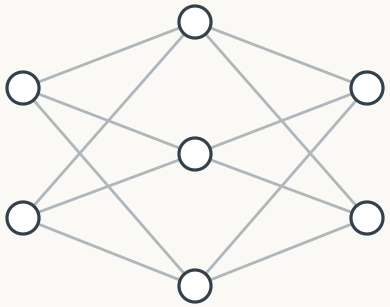


Books · websites · articles
Across many topics

Train from scratch

High compute cost
Long training time

Pre-trained model



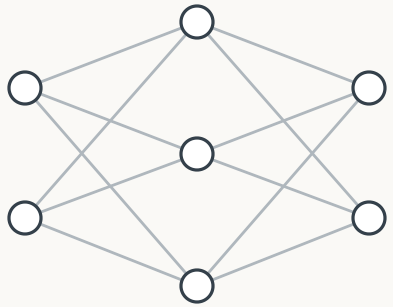
Language understanding
General knowledge

Reuse

Pre-train once. Reuse the base for many tasks.

Fine-tuning adapts the base to a task

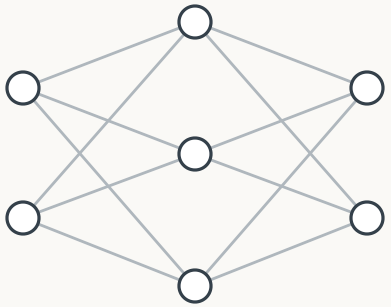
Pre-trained model



Reuse the same model
and its learned weights

Fine-tuning adapts the base to a task

Pre-trained model



Reuse the same model
and its learned weights



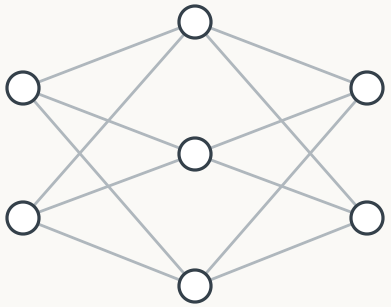
Task-specific data (e.g. math or code)

↓

Continue training

Fine-tuning adapts the base to a task

Pre-trained model



Reuse the same model
and its learned weights

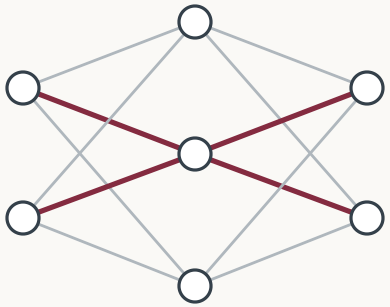


Task-specific data (e.g. math or code)

↓

Continue training

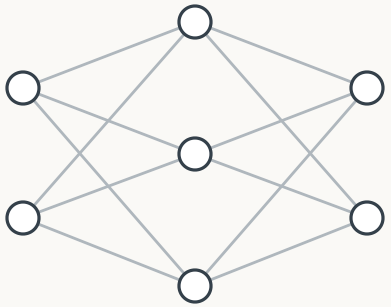
Task-adapted model



Better at the target task
e.g. writing code

Fine-tuning adapts the base to a task

Pre-trained model



Reuse the same model
and its learned weights

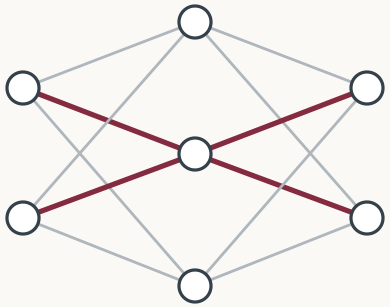


Task-specific data (e.g. math or code)

↓

Continue training

Task-adapted model



Better at the target task
e.g. writing code

Efficiency

Much less data and compute than training from scratch.

Fine-tuning can cause forgetting [10]

Illustrative example: fine-tuning to improve coding skills.

Fine-tuning can cause forgetting [10]

Illustrative example: fine-tuning to improve coding skills.

Before fine-tuning



Writes German essays correctly

Fine-tuning can cause forgetting [10]

Illustrative example: fine-tuning to improve coding skills.

Before fine-tuning



Writes German essays correctly

Train on code



Fine-tuning can cause forgetting [10]

Illustrative example: fine-tuning to improve coding skills.

Before fine-tuning



Writes German essays correctly

Train on code



After fine-tuning



Writes better code



May make grammatical mistakes in German

e.g. incorrect articles

Fine-tuning can cause forgetting [10]

Illustrative example: fine-tuning to improve coding skills.

Before fine-tuning



Writes German essays correctly

Train on code

After fine-tuning



Writes better code



May make grammatical mistakes in German

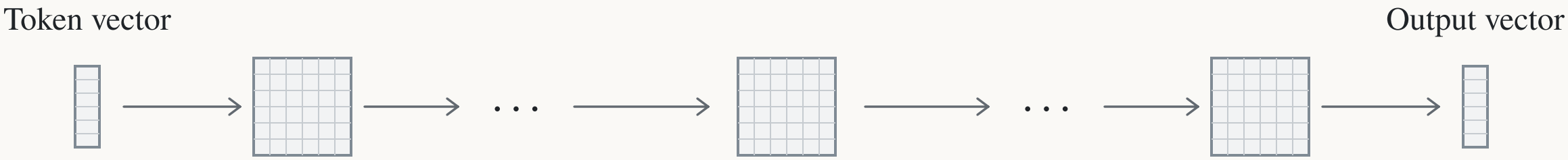
e.g. incorrect articles

Forgetting

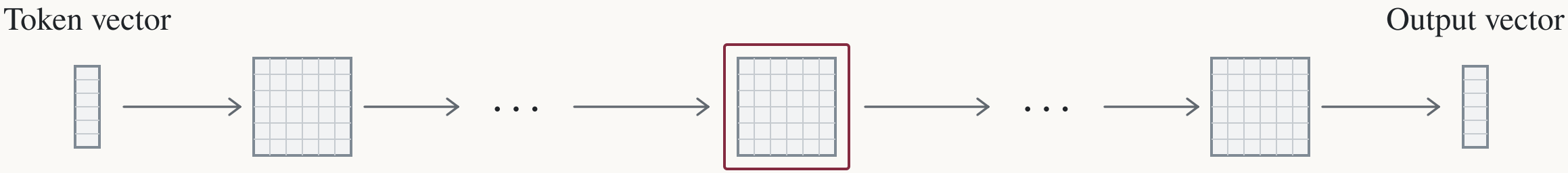
Losing previously learned knowledge or skills when learning a new task.

[10] Vu et al. (2022). *Overcoming Catastrophic Forgetting in Zero-Shot Cross-Lingual Generation.*

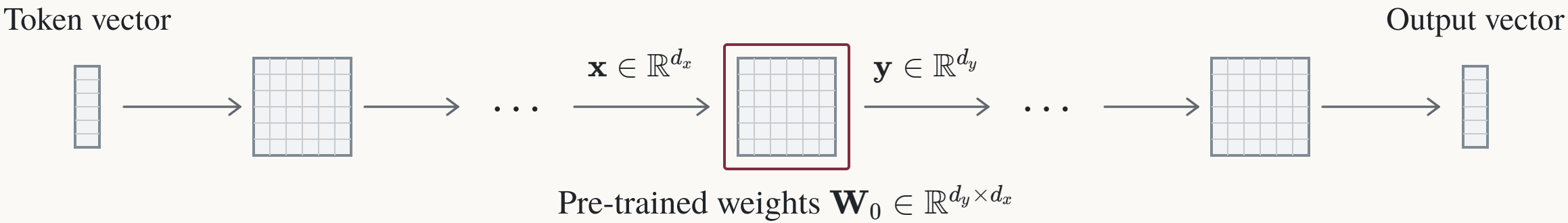
What is fine-tuning?



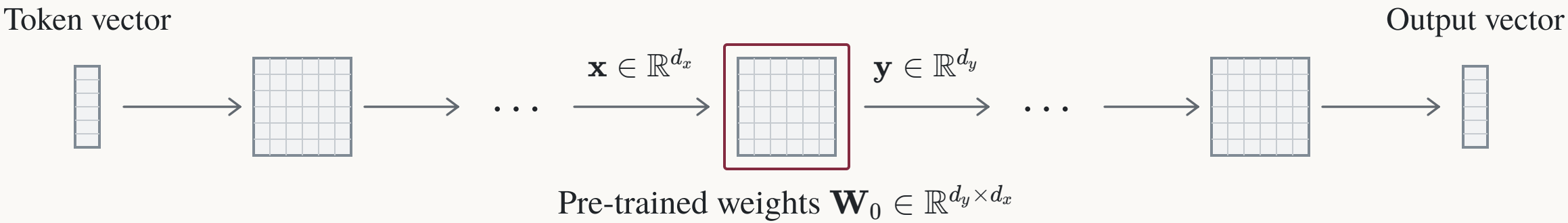
What is fine-tuning?



What is fine-tuning?

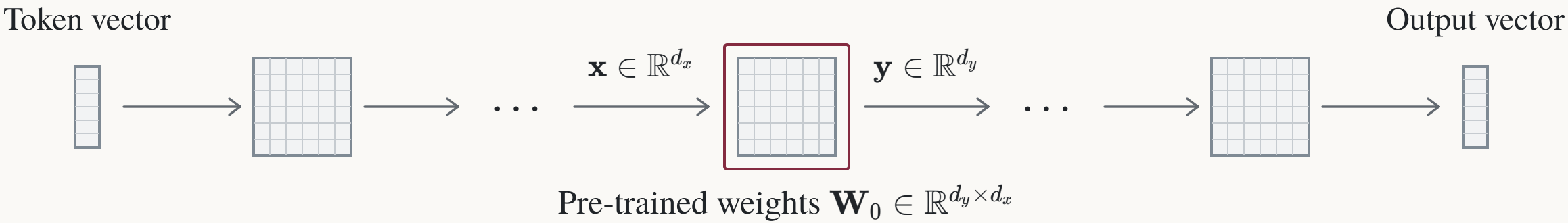


What is fine-tuning?



Focus on one matrix

What is fine-tuning?



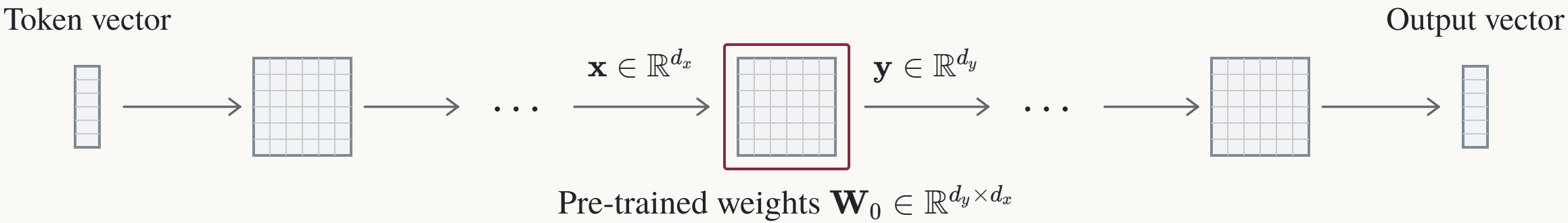
Focus on one matrix

Pre-trained

$$\mathbf{y}_{\text{pt}} = \mathbf{W}_0 \mathbf{x}_{\text{pt}}$$

$\mathbf{x}_{\text{pt}}$: pre-training data

What is fine-tuning?



Focus on one matrix

Pre-trained

$$\mathbf{y}_{\text{pt}} = \mathbf{W}_0 \mathbf{x}_{\text{pt}}$$

$\mathbf{x}_{\text{pt}}$: pre-training data

Learn correction $\boldsymbol{\Delta}$

New-task data

Fine-tuned

$$\mathbf{y}_{\text{ft}} = (\mathbf{W}_0 + \boldsymbol{\Delta})\mathbf{x}_{\text{ft}}$$

$\mathbf{x}_{\text{ft}}$: fine-tuning data

Goal

Adapt the pre-trained model to perform better on a new task.

What data do we use for fine-tuning and evaluation?

Fine-tuning



New-task
training data



Update the model

What data do we use for fine-tuning and evaluation?

Fine-tuning



New-task
training data

↓

Update the model

Evaluation



New-task
test data

↓

Measure adaptation

What data do we use for fine-tuning and evaluation?

Fine-tuning



New-task training data



Update the model

Evaluation



New-task test data



Measure adaptation



Pre-training-domain data



Measure retention

What data do we use for fine-tuning and evaluation?

Fine-tuning



New-task training data

↓

Update the model

Evaluation



New-task test data

↓

Measure adaptation



Pre-training-domain data

↓

Measure retention

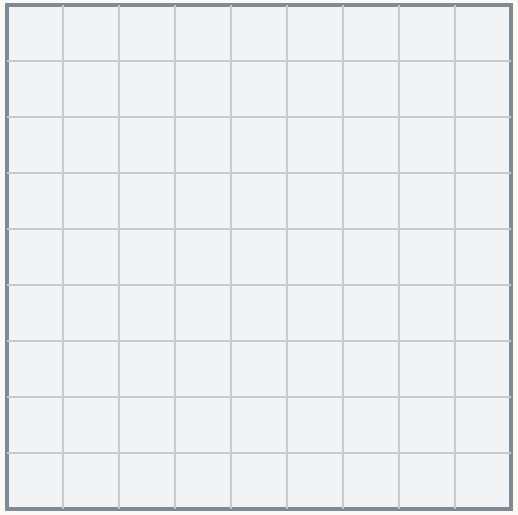
Data use

Pre-training-domain data is used for evaluation only.

Full fine-tuning (FFT): an unrestricted update [1]

One weight matrix in the network · $d_y \times d_x$ weights

$\mathbf{W}_0$

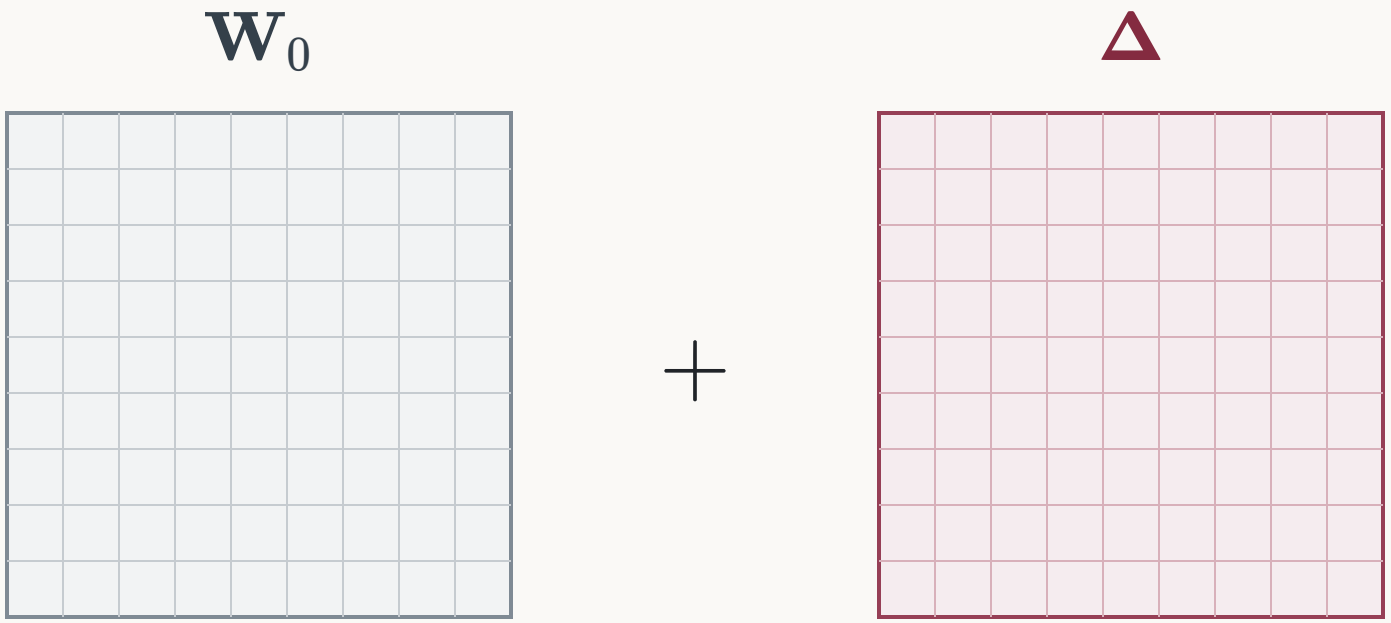


Starting weights

[1] Example of full fine-tuning: Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

Full fine-tuning (FFT): an unrestricted update [1]

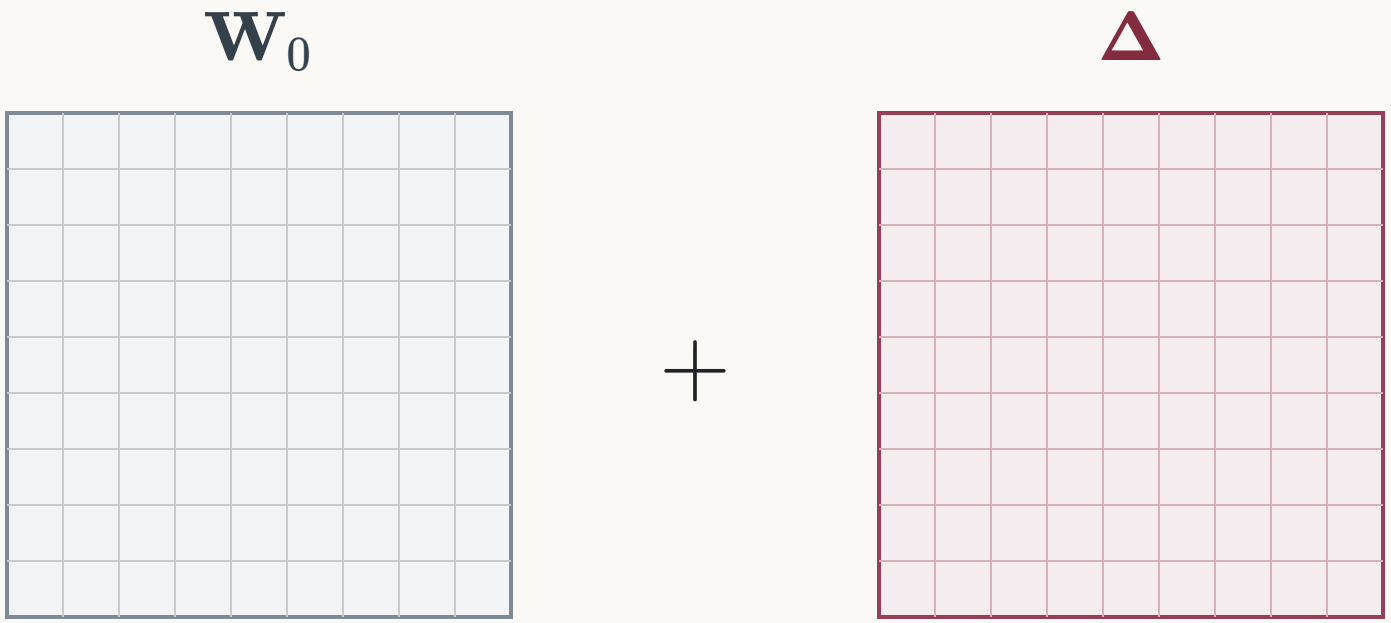
One weight matrix in the network · $d_y \times d_x$ weights



[1] Example of full fine-tuning: Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.

Full fine-tuning (FFT): an unrestricted update [1]

One weight matrix in the network · $d_y \times d_x$ weights

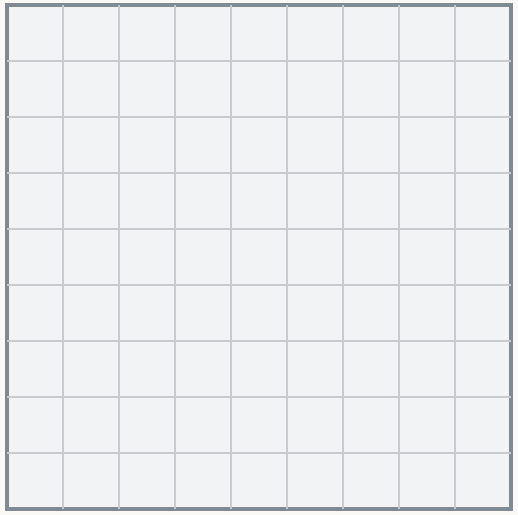


Trainable parameters	Example: $d_x = d_y = 4096$
$d_x d_y$	16,777,216

[1] Example of full fine-tuning: Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.*

LoRA learns a low-rank matrix update [2]

$\mathbf{W}_0$

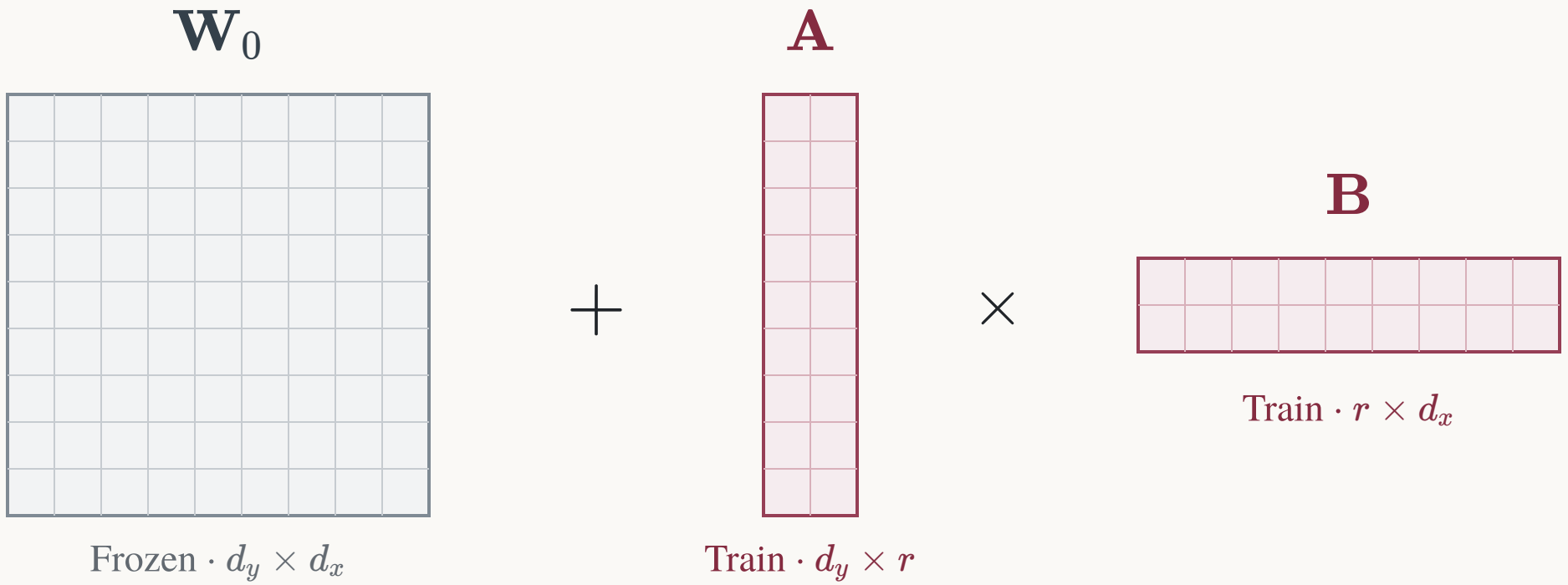


Frozen · $d_y \times d_x$

[2] Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR.

LoRA learns a low-rank matrix update [2]

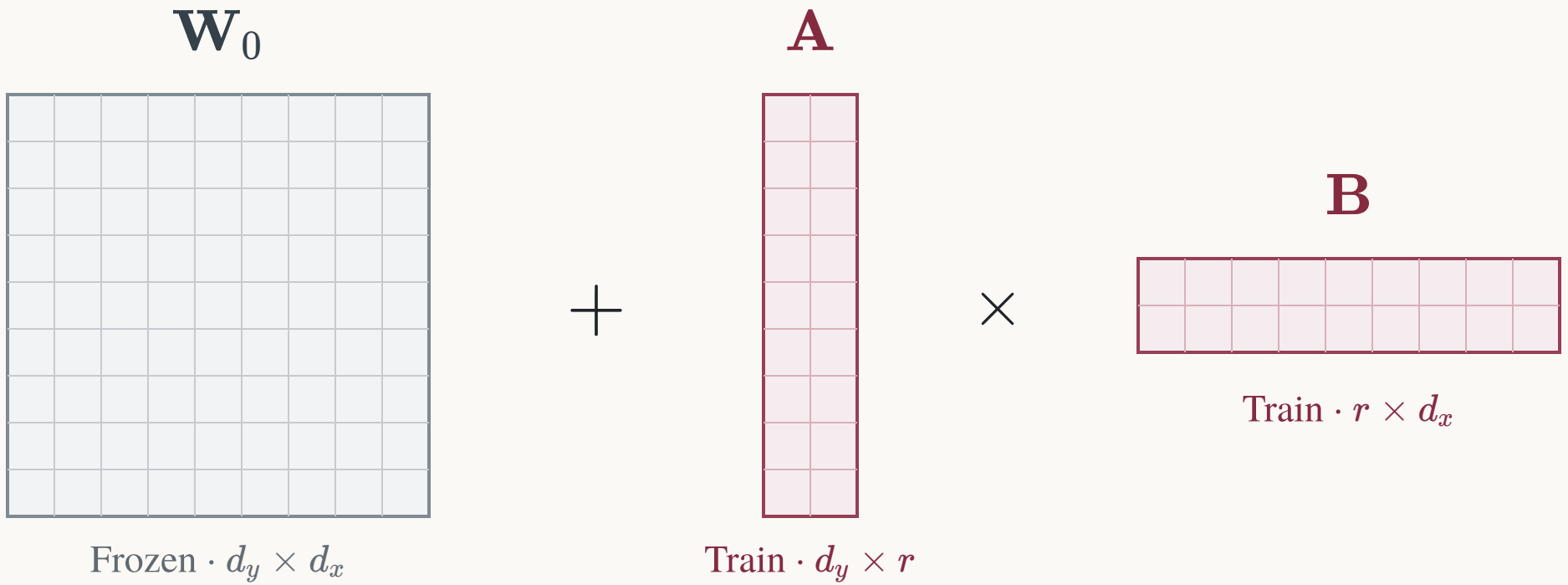
Factorize the update: $\mathbf{\Delta} = \mathbf{AB}, \quad \text{rank}(\mathbf{\Delta}) \leq r$



[2] Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR.

LoRA learns a low-rank matrix update [2]

Factorize the update: $\mathbf{\Delta} = \mathbf{AB}, \quad \text{rank}(\mathbf{\Delta}) \leq r$



Trainable parameters	$r = 16$
$r(d_x + d_y)$, with $r \ll \min(d_x, d_y)$	131,072 128× fewer

[2] Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR.

Why can FFT and LoRA forget?

After updating $\mathbf{W}_0$ using new task data:

$$\mathbf{W}_0 \longrightarrow \mathbf{W}_0 + \Delta$$

Why can FFT and LoRA forget?

After updating $\mathbf{W}_0$ using new task data:

$$\mathbf{W}_0 \longrightarrow \mathbf{W}_0 + \boldsymbol{\Delta}$$

New-task input $\mathbf{x}_{\text{ft}}$

After fine-tuning

$$\mathbf{y}_{\text{ft}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{ft}}$$

Correction learned for the new task

Why can FFT and LoRA forget?

After updating $\mathbf{W}_0$ using new task data:

$$\mathbf{W}_0 \longrightarrow \mathbf{W}_0 + \boldsymbol{\Delta}$$

New-task input $\mathbf{x}_{\text{ft}}$

After fine-tuning

$$\mathbf{y}_{\text{ft}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{ft}}$$

Correction learned for the new task

Earlier input $\mathbf{x}_{\text{pt}}$

Desired output: $\mathbf{y}_{\text{pt}} = \mathbf{W}_0\mathbf{x}_{\text{pt}}$

Why can FFT and LoRA forget?

After updating $\mathbf{W}_0$ using new task data:

$$\mathbf{W}_0 \longrightarrow \mathbf{W}_0 + \boldsymbol{\Delta}$$

New-task input $\mathbf{x}_{\text{ft}}$

After fine-tuning

$$\mathbf{y}_{\text{ft}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{ft}}$$

Correction learned for the new task

Earlier input $\mathbf{x}_{\text{pt}}$

Desired output: $\mathbf{y}_{\text{pt}} = \mathbf{W}_0\mathbf{x}_{\text{pt}}$

After fine-tuning

$$\mathbf{y}_{\text{pt}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{pt}}$$

The same update can disturb this answer.

Why can FFT and LoRA forget?

After updating $\mathbf{W}_0$ using new task data:

$$\mathbf{W}_0 \longrightarrow \mathbf{W}_0 + \boldsymbol{\Delta}$$

New-task input $\mathbf{x}_{\text{ft}}$

After fine-tuning

$$\mathbf{y}_{\text{ft}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{ft}}$$

Correction learned for the new task

Earlier input $\mathbf{x}_{\text{pt}}$

Desired output: $\mathbf{y}_{\text{pt}} = \mathbf{W}_0\mathbf{x}_{\text{pt}}$

After fine-tuning

$$\mathbf{y}_{\text{pt}} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}\right)\mathbf{x}_{\text{pt}}$$

The same update can disturb this answer.

Retention error

$$\left\|(\mathbf{W}_0 + \boldsymbol{\Delta})\mathbf{x}_{\text{pt}} - \mathbf{W}_0\mathbf{x}_{\text{pt}}\right\|^2 = \|\boldsymbol{\Delta}\mathbf{x}_{\text{pt}}\|^2$$

New-task training does not force this error to be zero.

Our method: DISeL — the idea [3]

Make $\boldsymbol{\Delta}$ input-dependent.

$$\boldsymbol{\Delta} \longrightarrow \boldsymbol{\Delta}(\mathbf{x})$$

[3] Zindari et al. (2026). *Learning When to Adapt.*

Our method: DISeL — the idea [3]

Make $\boldsymbol{\Delta}$ input-dependent.

$$\boldsymbol{\Delta} \longrightarrow \boldsymbol{\Delta}(\mathbf{x})$$

Desired behavior

New-task input $\mathbf{x}_{\text{ft}}$

A correction is needed.

$$\mathbf{y} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}(\mathbf{x}_{\text{ft}})\right)\mathbf{x}_{\text{ft}}$$

Allow the correction

[3] Zindari et al. (2026). *Learning When to Adapt.*

Our method: DISeL — the idea [3]

Make $\boldsymbol{\Delta}$ input-dependent.

$$\boldsymbol{\Delta} \longrightarrow \boldsymbol{\Delta}(\mathbf{x})$$

Desired behavior

New-task input $\mathbf{x}_{\text{ft}}$

A correction is needed.

$$\mathbf{y} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}(\mathbf{x}_{\text{ft}})\right)\mathbf{x}_{\text{ft}}$$

Allow the correction

Earlier input $\mathbf{x}_{\text{pt}}$

The original answer is correct.

$$\mathbf{y} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}(\mathbf{x}_{\text{pt}})\right)\mathbf{x}_{\text{pt}}$$

$$\boldsymbol{\Delta}(\mathbf{x}_{\text{pt}}) \approx \mathbf{0}$$

Suppress the correction

[3] Zindari et al. (2026). *Learning When to Adapt.*

Our method: DISeL — the idea [3]

Make $\boldsymbol{\Delta}$ input-dependent.

$$\boldsymbol{\Delta} \longrightarrow \boldsymbol{\Delta}(\mathbf{x})$$

Desired behavior

New-task input $\mathbf{x}_{\text{ft}}$

A correction is needed.

$$\mathbf{y} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}(\mathbf{x}_{\text{ft}})\right)\mathbf{x}_{\text{ft}}$$

Allow the correction

Earlier input $\mathbf{x}_{\text{pt}}$

The original answer is correct.

$$\mathbf{y} = \left(\mathbf{W}_0 + \boldsymbol{\Delta}(\mathbf{x}_{\text{pt}})\right)\mathbf{x}_{\text{pt}}$$

$$\boldsymbol{\Delta}(\mathbf{x}_{\text{pt}}) \approx \mathbf{0}$$

Suppress the correction

Training

Learn the correction and when to apply it, using only new-task data.

[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: allow correction for inputs from fine-tuning data [3]

$$\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A} \qquad \mathbf{B}$$

[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: allow correction for inputs from fine-tuning data [3]

$$\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A}\,\mathbf{G}(\mathbf{x})\,\mathbf{B} \qquad \mathbf{G}(\mathbf{x}) \in \mathbb{R}^{r \times r}$$

[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: allow correction for inputs from fine-tuning data [3]

$$\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A}\,\mathbf{G}(\mathbf{x})\,\mathbf{B} \qquad \mathbf{G}(\mathbf{x}) \in \mathbb{R}^{r\times r}$$

For $\mathbf{x} = \mathbf{x}_{\text{ft}}$ from fine-tuning data, we want $\mathbf{G}(\mathbf{x}_{\text{ft}}) = \mathbf{I}$.

$$\boldsymbol{\Delta}(\mathbf{x}_{\text{ft}}) = \mathbf{AB} \quad \Longrightarrow \quad \mathbf{y} = (\mathbf{W}_0 + \mathbf{AB})\mathbf{x}_{\text{ft}} \quad \text{LoRA update}$$

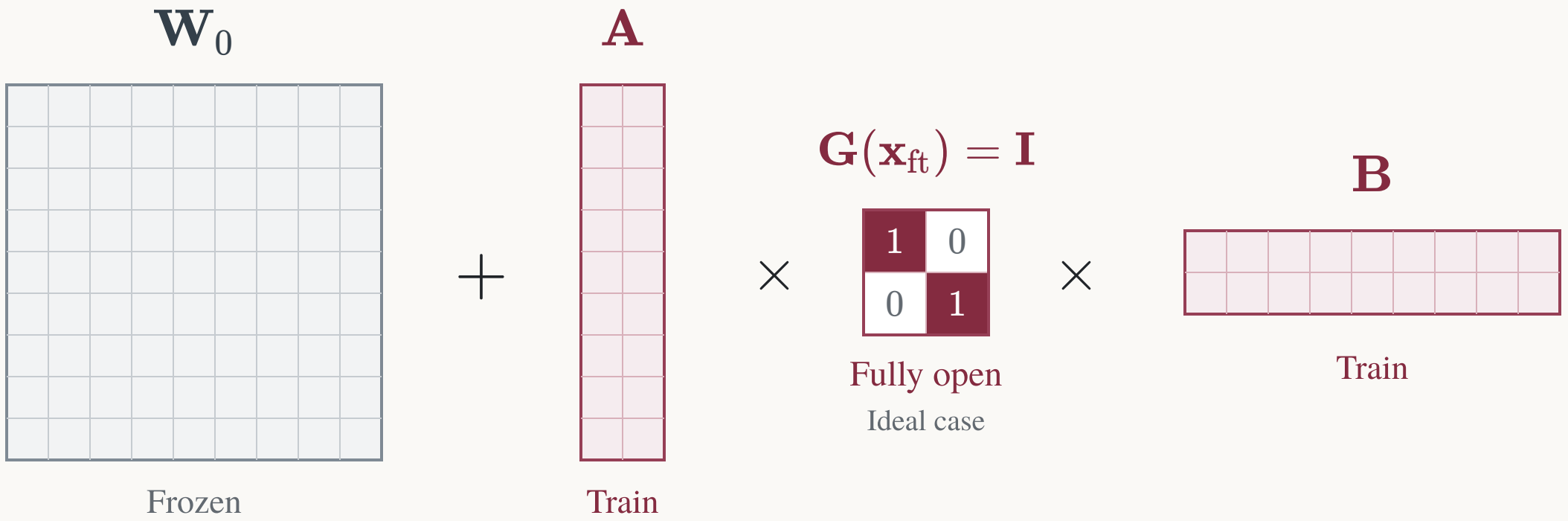
[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: allow correction for inputs from fine-tuning data [3]

$$\Delta(\mathbf{x}) = \mathbf{A}\,\mathbf{G}(\mathbf{x})\,\mathbf{B} \qquad \mathbf{G}(\mathbf{x}) \in \mathbb{R}^{r\times r}$$

For $\mathbf{x} = \mathbf{x}_{\text{ft}}$ from fine-tuning data, we want $\mathbf{G}(\mathbf{x}_{\text{ft}}) = \mathbf{I}$.

$$\Delta(\mathbf{x}_{\text{ft}}) = \mathbf{AB} \quad \Longrightarrow \quad \mathbf{y} = (\mathbf{W}_0 + \mathbf{AB})\mathbf{x}_{\text{ft}} \quad \text{LoRA update}$$



[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: suppress correction for inputs from pre-training data [3]

$$\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B} \qquad \mathbf{G}(\mathbf{x}) \in \mathbb{R}^{r \times r}$$

For $\mathbf{x} = \mathbf{x}_{\text{pt}}$ from the pre-training domain, we want $\mathbf{G}(\mathbf{x}_{\text{pt}}) = \mathbf{0}$.

$$\boldsymbol{\Delta}(\mathbf{x}_{\text{pt}}) = \mathbf{0} \quad \Longrightarrow \quad \mathbf{y} = \mathbf{W}_0\mathbf{x}_{\text{pt}} \quad \text{Pre-trained output}$$

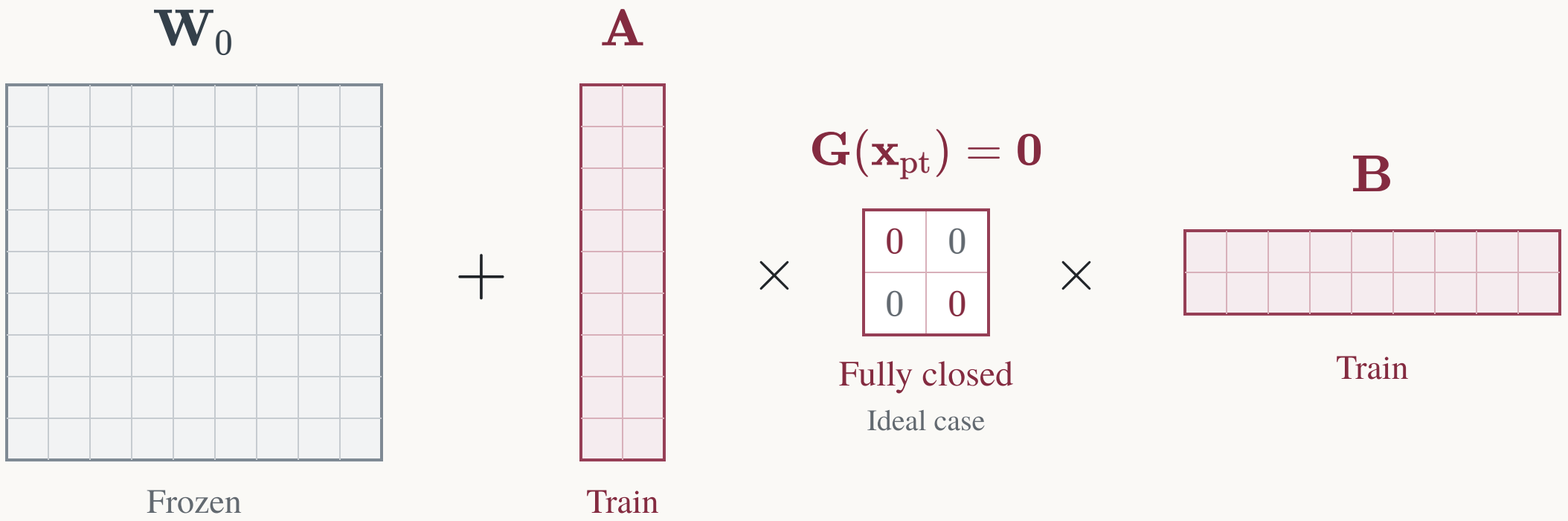
[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: suppress correction for inputs from pre-training data [3]

$$\Delta(\mathbf{x}) = \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B} \qquad \mathbf{G}(\mathbf{x}) \in \mathbb{R}^{r \times r}$$

For $\mathbf{x} = \mathbf{x}_{\text{pt}}$ from the pre-training domain, we want $\mathbf{G}(\mathbf{x}_{\text{pt}}) = \mathbf{0}$.

$$\Delta(\mathbf{x}_{\text{pt}}) = \mathbf{0} \implies \mathbf{y} = \mathbf{W}_0\mathbf{x}_{\text{pt}} \quad \text{Pre-trained output}$$



[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: learn the gates from the input [3]

$$\mathbf{y} = \left(\mathbf{W}_0 + \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B}\right)\mathbf{x}$$

[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: learn the gates from the input [3]

$$\mathbf{y} = \left(\mathbf{W}_0 + \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B}\right)\mathbf{x}$$

$$\mathbf{G}(\mathbf{x}) = \operatorname{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right), \quad \mathbf{w}_g \in \mathbb{R}^{r \times d_x}, \quad \mathbf{b} \in \mathbb{R}^r$$

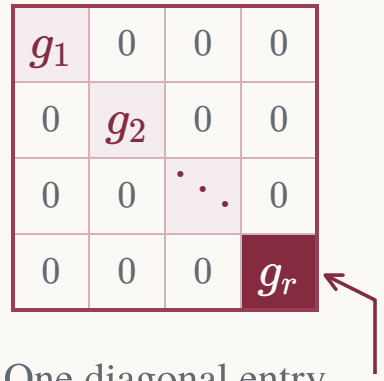
[3] Zindari et al. (2026). *Learning When to Adapt.*

DISeL: learn the gates from the input [3]

$$\mathbf{y} = \left(\mathbf{W}_0 + \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B}\right)\mathbf{x}$$

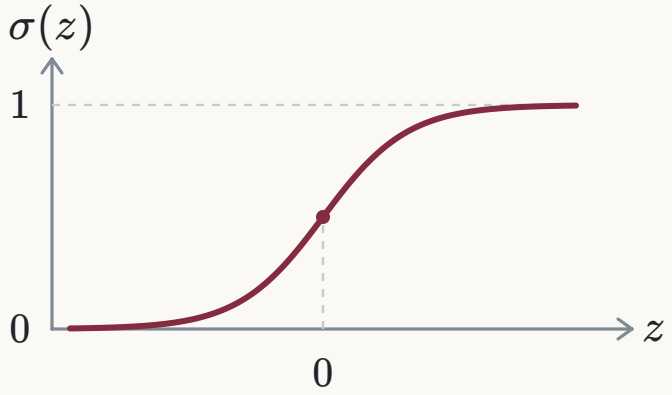
$$\mathbf{G}(\mathbf{x}) = \operatorname{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right), \quad \mathbf{W}_g \in \mathbb{R}^{r \times d_x}, \quad \mathbf{b} \in \mathbb{R}^r$$

Diagonal gates



$$g_r(\mathbf{x}) = \sigma\left((\mathbf{W}_g\mathbf{x})_r + b_r\right)$$

Sigmoid



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

[3] Zindari et al. (2026). *Learning When to Adapt.*

Start nearly closed. Learn when to open.

$$\mathbf{G}(\mathbf{x}) = \text{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right)$$

Start nearly closed. Learn when to open.

$$\mathbf{G}(\mathbf{x}) = \text{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right)$$

$$\mathbf{W}_g = \mathbf{0}, \quad b_i = -3 \quad \longrightarrow \quad \sigma(-3) \approx 0.047$$

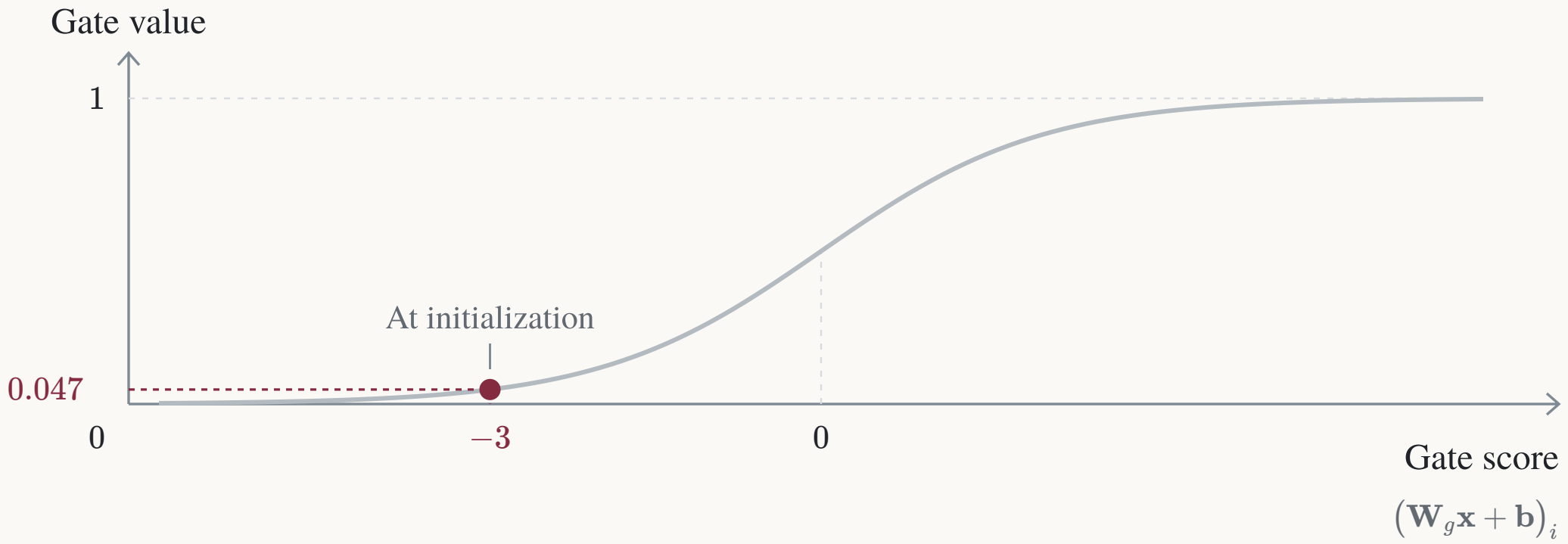
Initial bias: a hyperparameter.

Start nearly closed. Learn when to open.

$$\mathbf{G}(\mathbf{x}) = \text{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right) \qquad \mathbf{W}_g = \mathbf{0}, \quad b_i = -3 \quad \longrightarrow \quad \sigma(-3) \approx 0.047$$

Initial bias: a hyperparameter.

Initially: $\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B} \approx 0.047\,\mathbf{A}\mathbf{B}$ $\longrightarrow$ Update scaled down

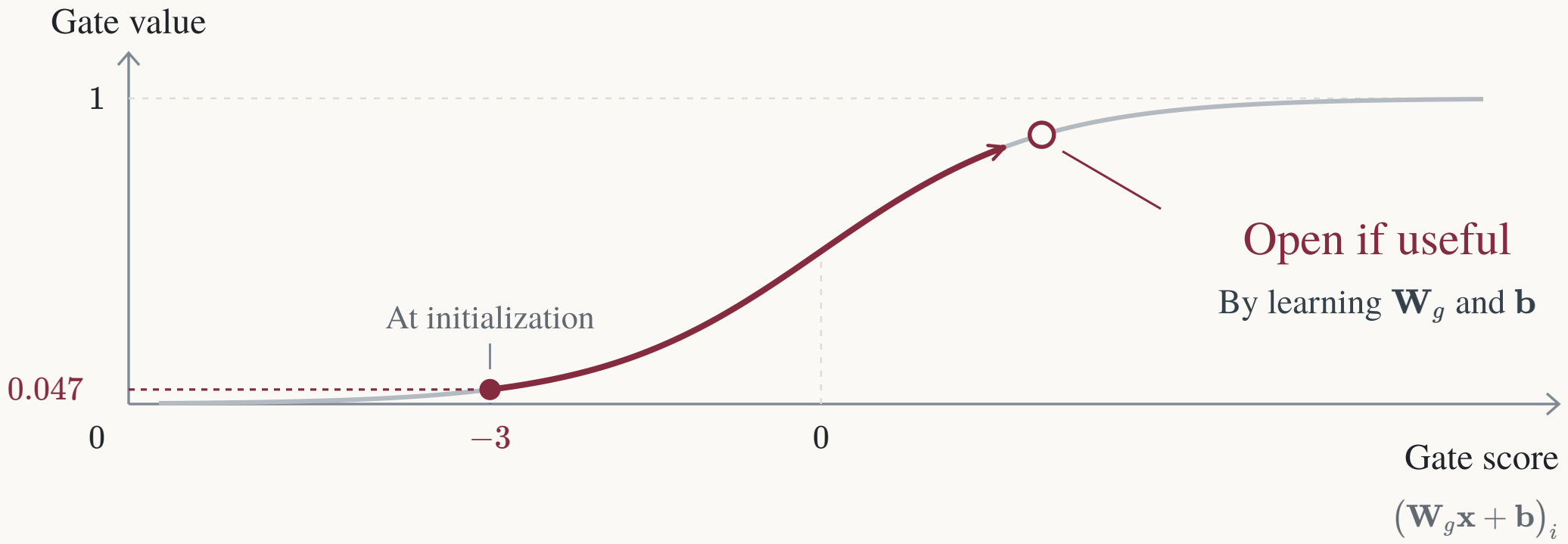


Start nearly closed. Learn when to open.

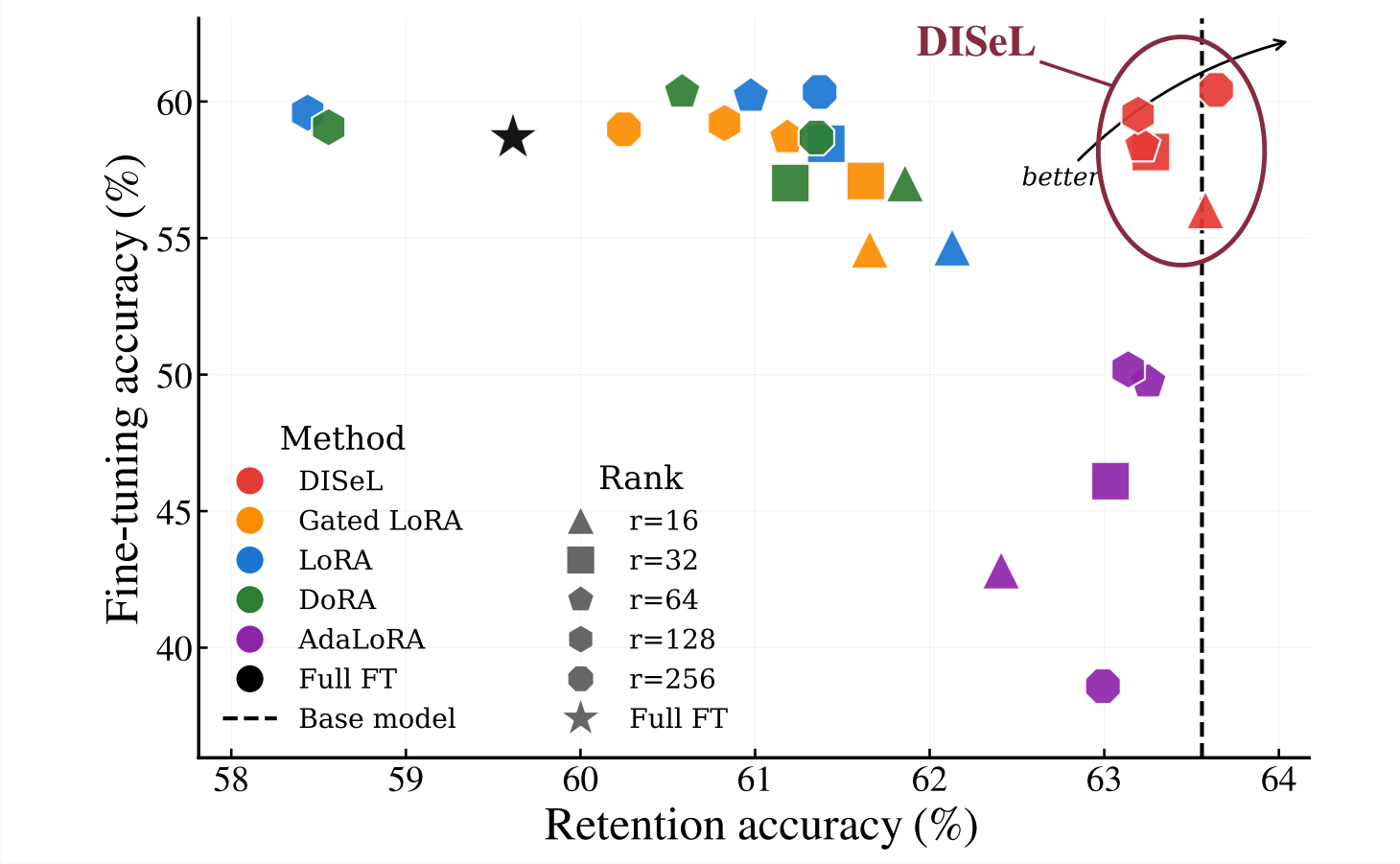
$$\mathbf{G}(\mathbf{x}) = \operatorname{diag}\left(\sigma(\mathbf{W}_g\mathbf{x} + \mathbf{b})\right) \qquad \mathbf{W}_g = \mathbf{0}, \quad b_i = -3 \longrightarrow \sigma(-3) \approx 0.047$$

Initial bias: a hyperparameter.

Initially: $\boldsymbol{\Delta}(\mathbf{x}) = \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B} \approx 0.047\,\mathbf{A}\mathbf{B}$ ⟶ Update scaled down

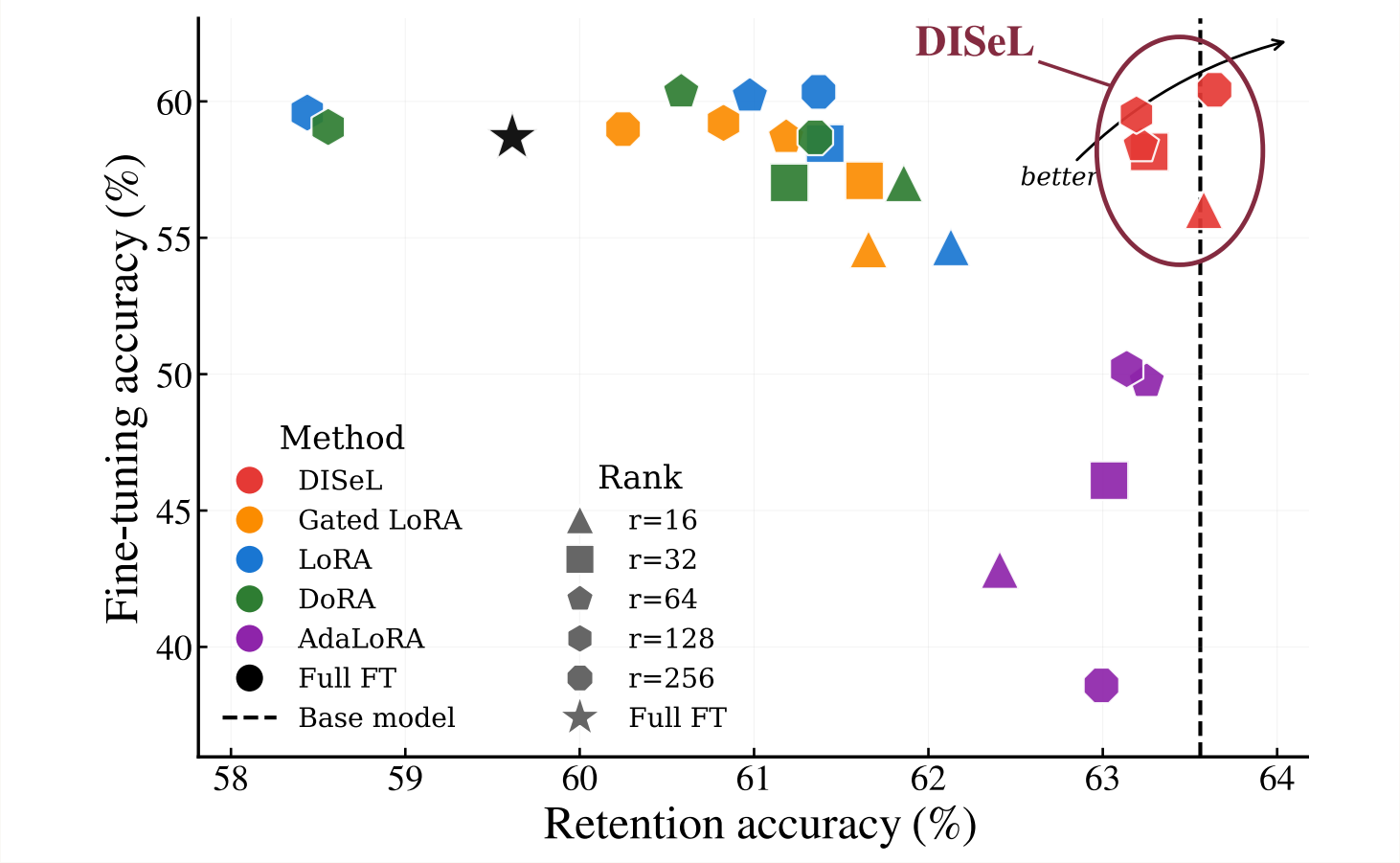


Math learning with retention near the base model



Llama 2-7B · Math fine-tuning

Math learning with retention near the base model



Llama 2-7B · Math fine-tuning

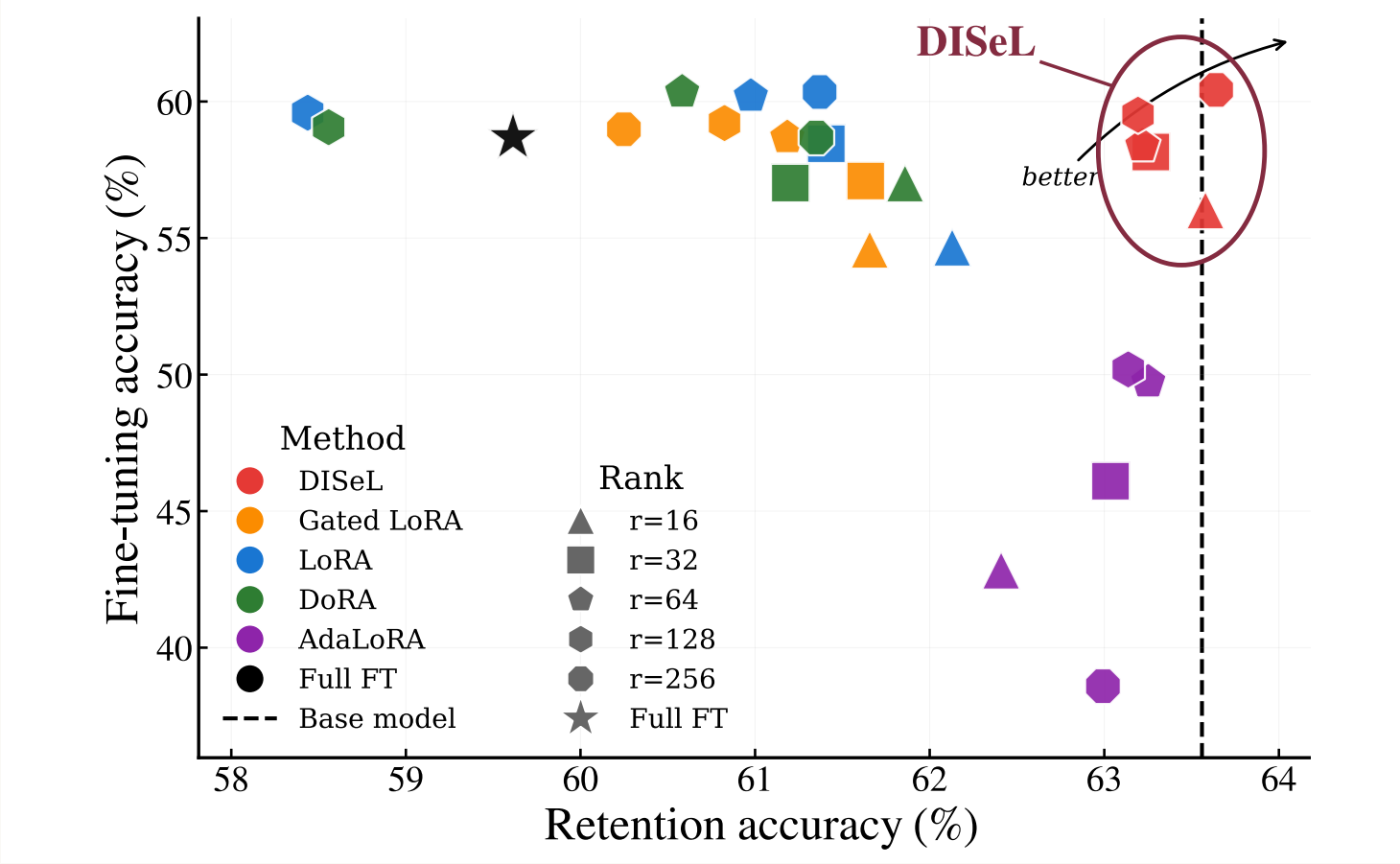
$\approx 60\%$

GSM8K accuracy

$\approx 63.3\%$

Mean accuracy on 14 retention benchmarks

Math learning with retention near the base model



Llama 2-7B · Math fine-tuning

$\approx 60\%$

GSM8K accuracy

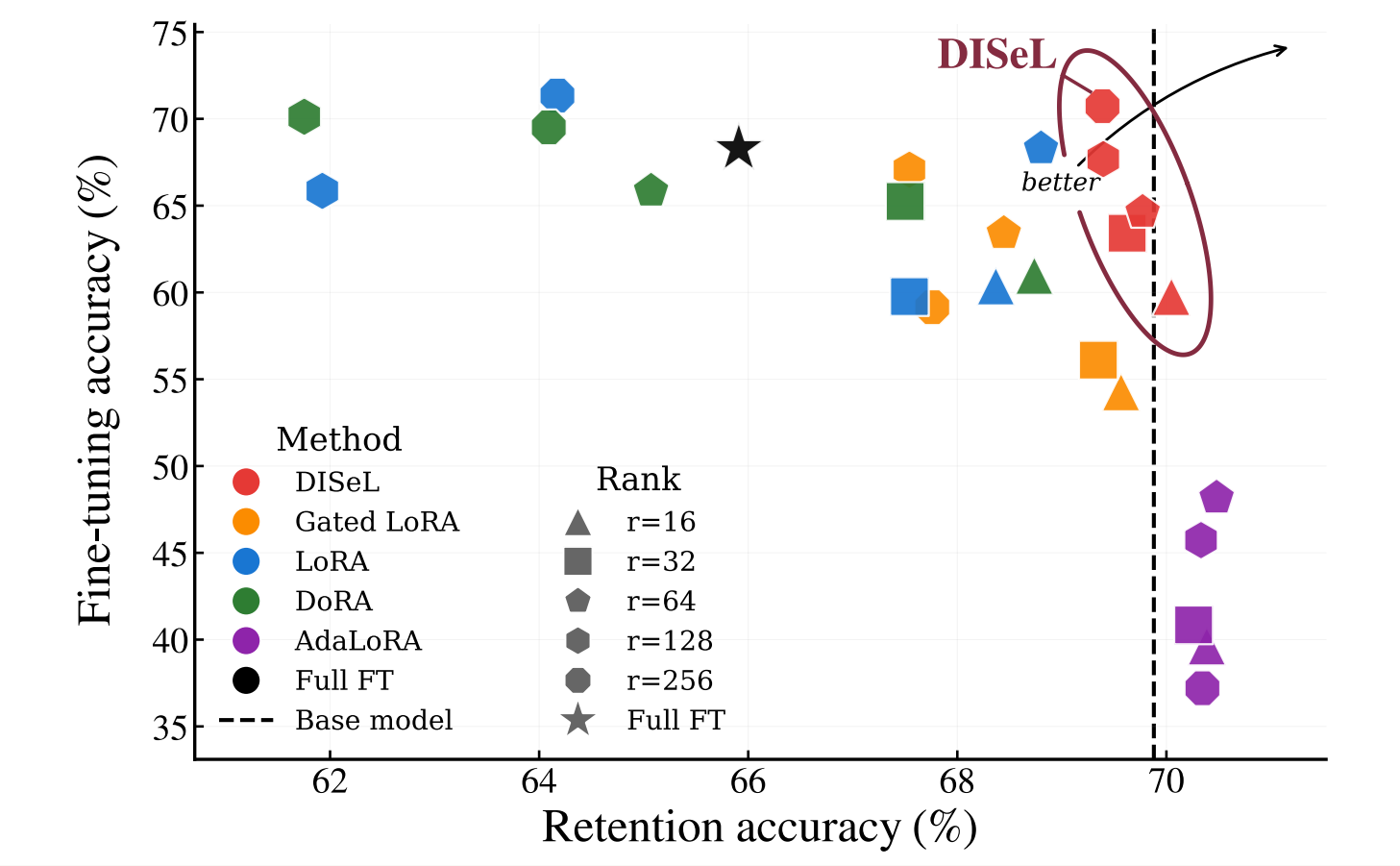
$\approx 63.3\%$

Mean accuracy on 14 retention benchmarks

Main result

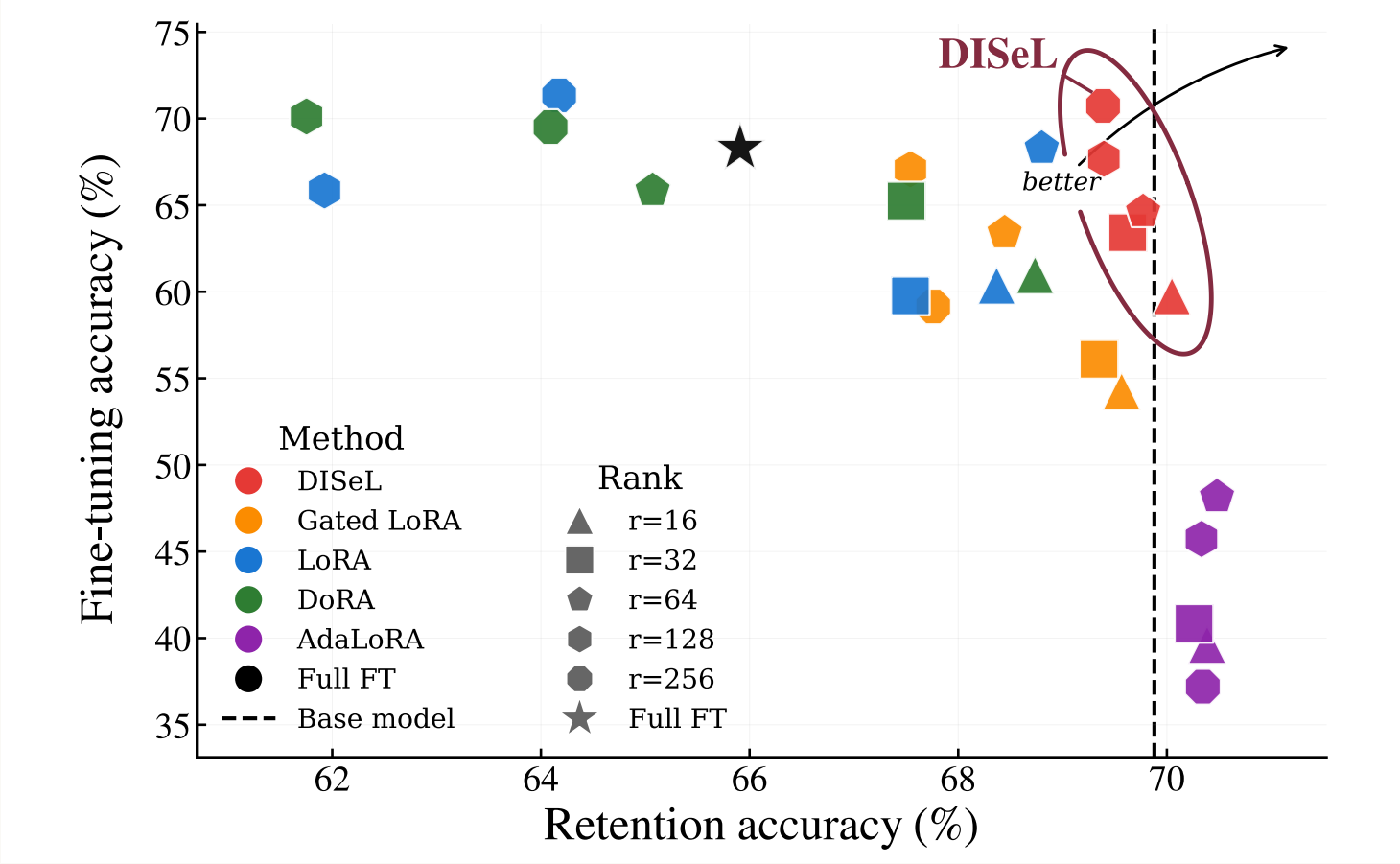
DISeL offers the best of both worlds: strong adaptation and strong retention.

The same tradeoff improves on code generation



Mistral-7B · Code fine-tuning

The same tradeoff improves on code generation



Mistral-7B · Code fine-tuning

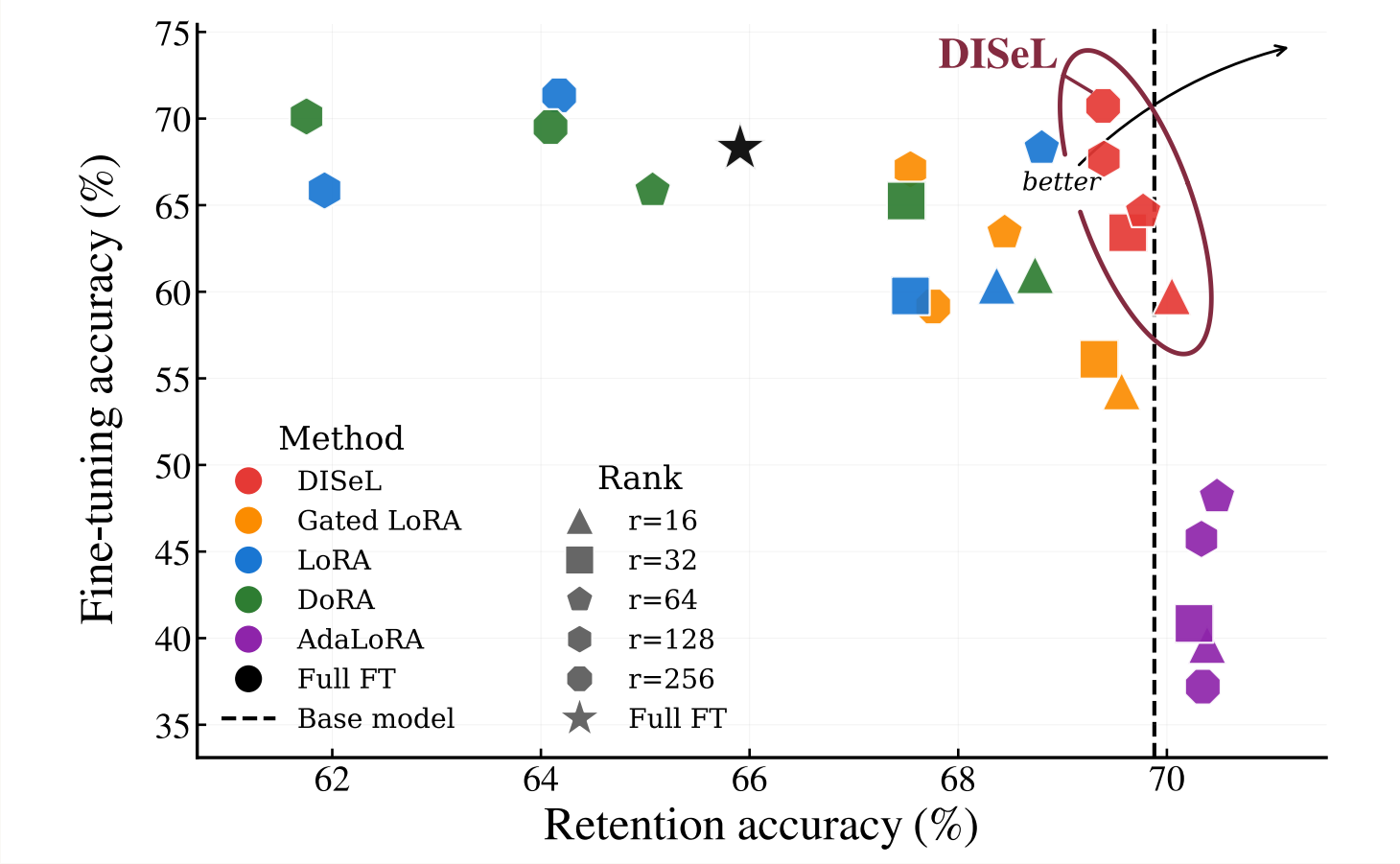
60–70%

HumanEval pass@1

≈ 70%

Mean accuracy on 14 retention benchmarks

The same tradeoff improves on code generation



Mistral-7B · Code fine-tuning

60–70%

HumanEval pass@1

≈ 70%

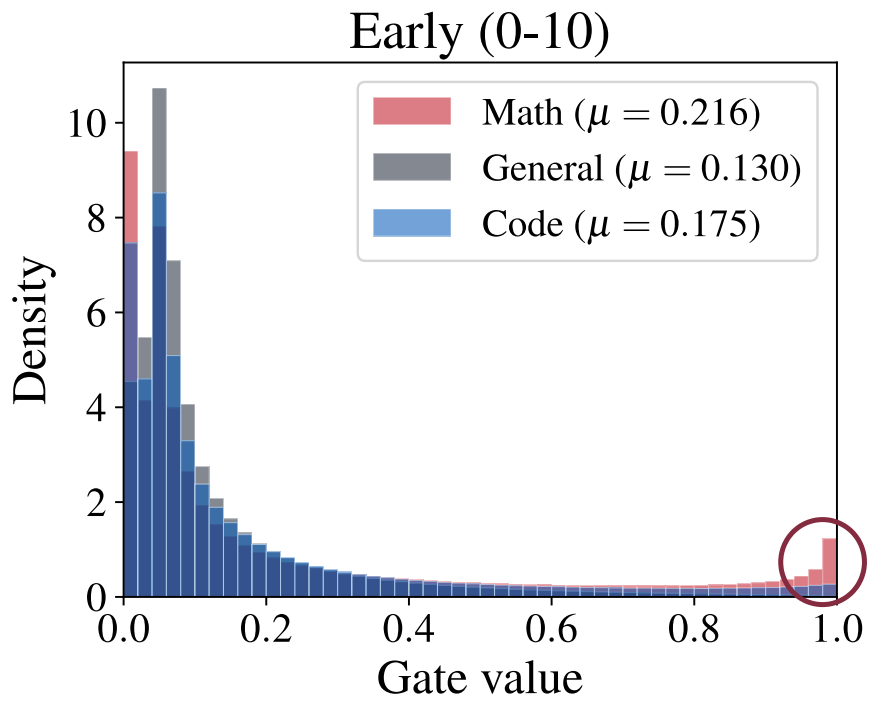
Mean accuracy on 14 retention benchmarks

Adaptation matters too

AdaLoRA preserves earlier abilities but struggles to learn the new task.

The learned gates respond to the input and layer

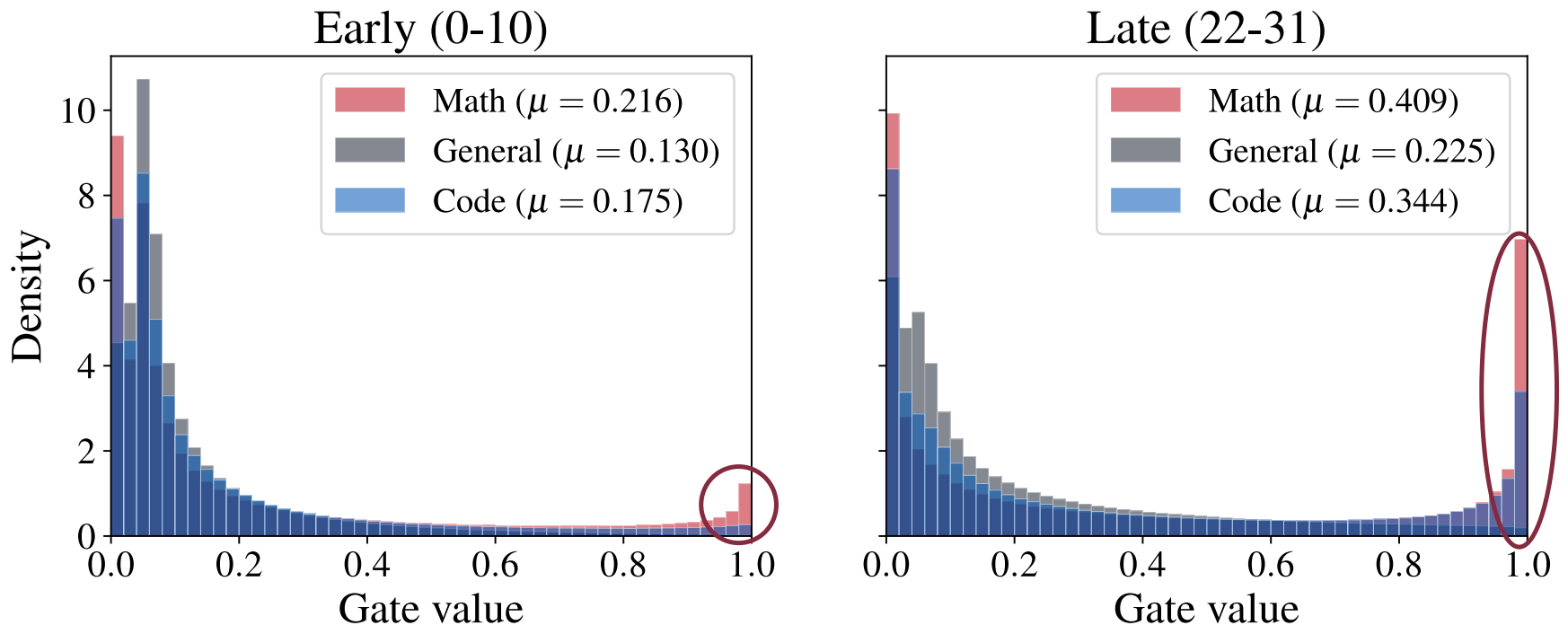
Rank-32 Llama math adapter · Held-out math, general text, and code



$$\text{Density} = \frac{\text{fraction of values in bin}}{\text{bin width}}$$

The learned gates respond to the input and layer

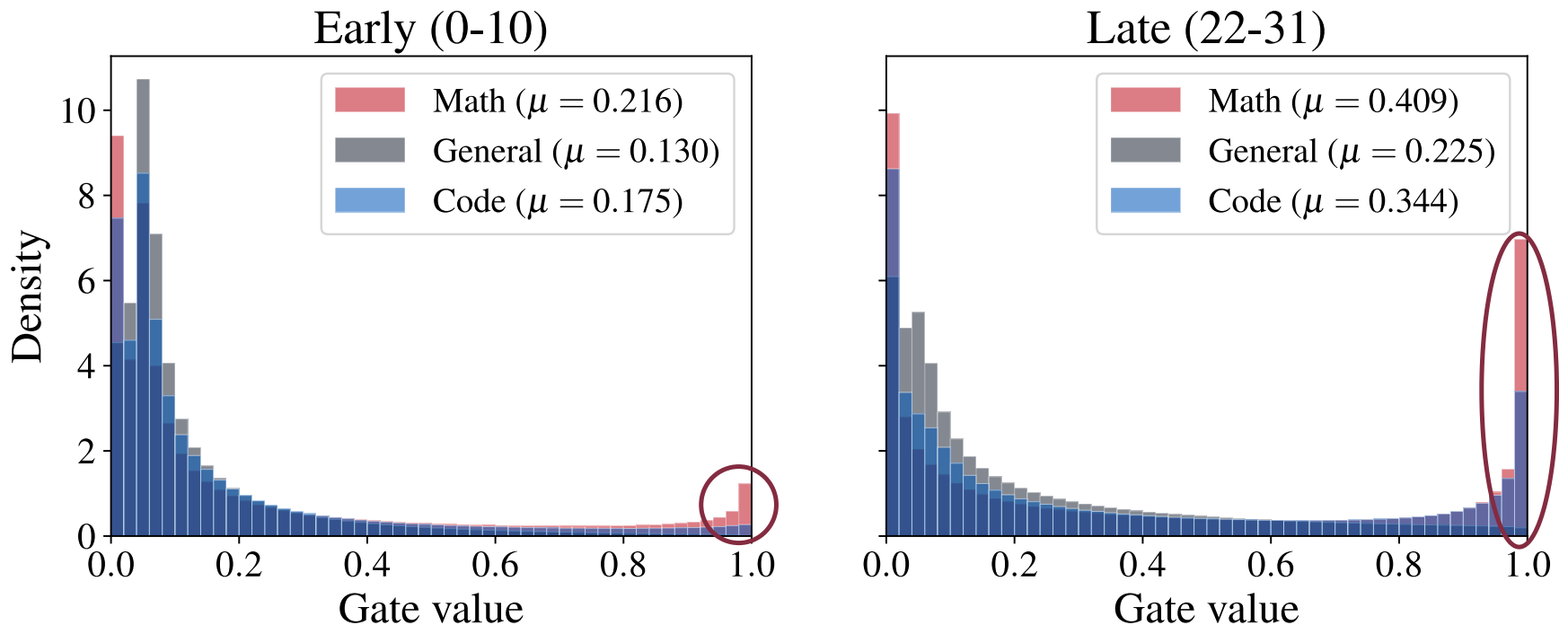
Rank-32 Llama math adapter · Held-out math, general text, and code



$$\text{Density} = \frac{\text{fraction of values in bin}}{\text{bin width}}$$

The learned gates respond to the input and layer

Rank-32 Llama math adapter · Held-out math, general text, and code

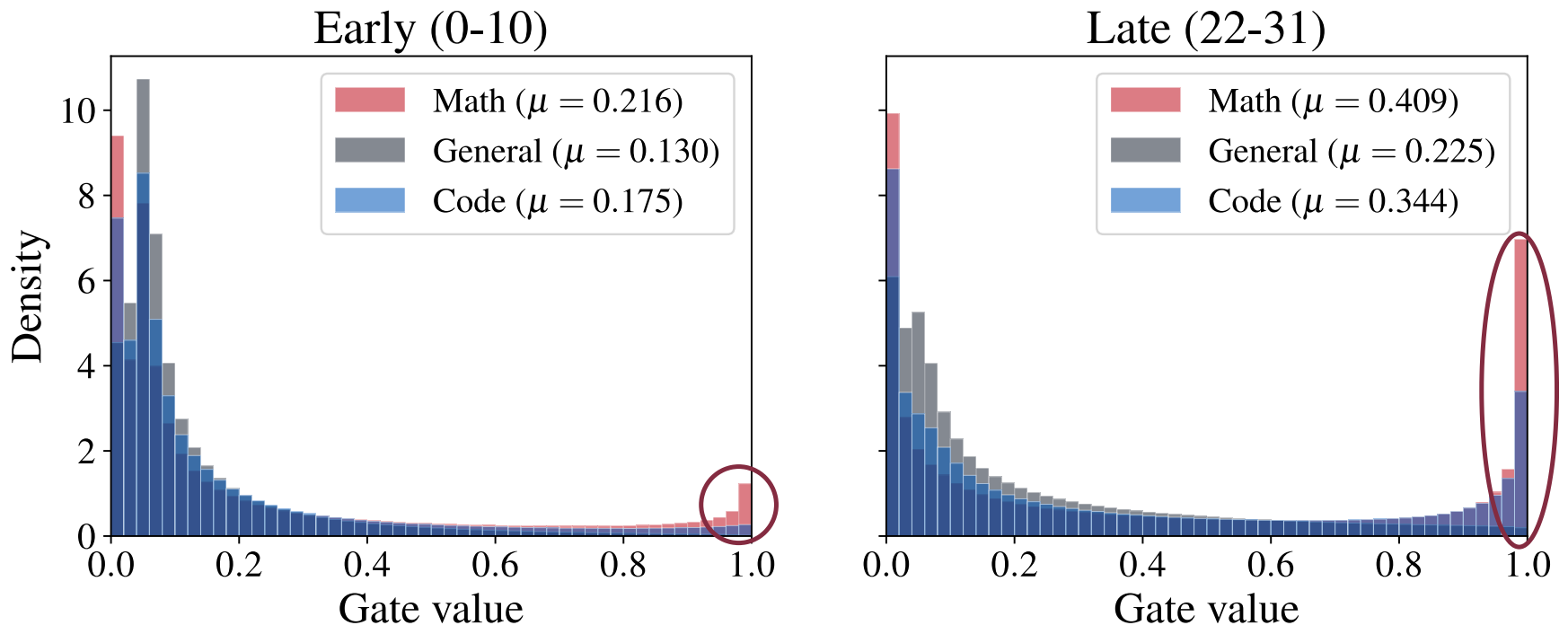


Input selectivity: math opens gates more than general text.

$$\text{Density} = \frac{\text{fraction of values in bin}}{\text{bin width}}$$

The learned gates respond to the input and layer

Rank-32 Llama math adapter · Held-out math, general text, and code

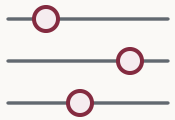


Input selectivity: math opens gates more than general text.

Layer selectivity: larger gaps in later layers.

$$\text{Density} = \frac{\text{fraction of values in bin}}{\text{bin width}}$$

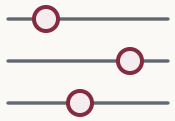
DISeL: learn when to adapt



Input-dependent

Adjusts the update
to each input.

DISeL: learn when to adapt



Input-dependent

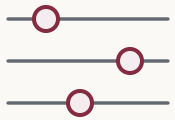
Adjusts the update to each input.



Training data only

No pre-training data needed.

DISeL: learn when to adapt



Input-dependent

Adjusts the update to each input.



Training data only

No pre-training data needed.



Less forgetting

Keeps strong performance on the new task.

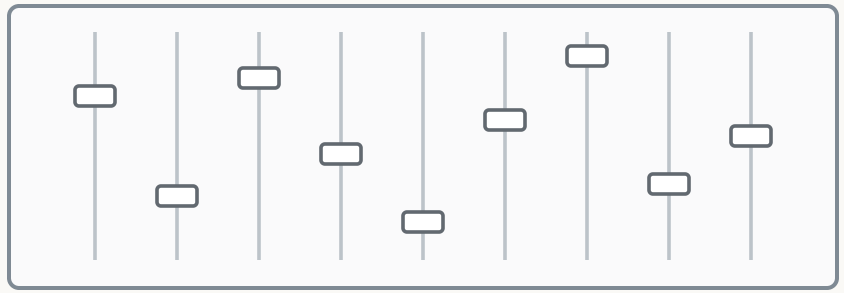
LoRA vs. Full Fine-Tuning

Can fewer trainable parameters lead to better accuracy?

LoRA vs. Full Fine-Tuning

Can fewer trainable parameters lead to better accuracy?

Full fine-tuning

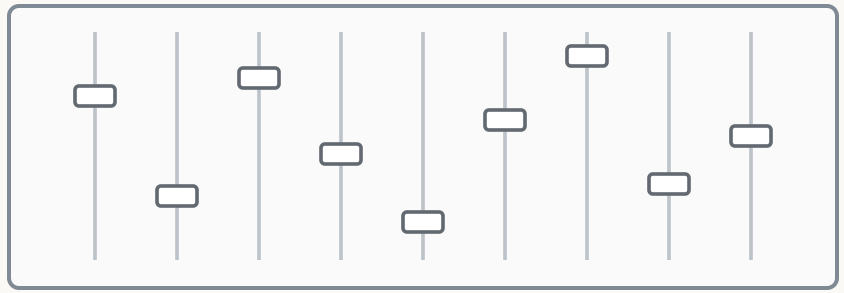


More freedom to change.
Can also fit noise in the data.

LoRA vs. Full Fine-Tuning

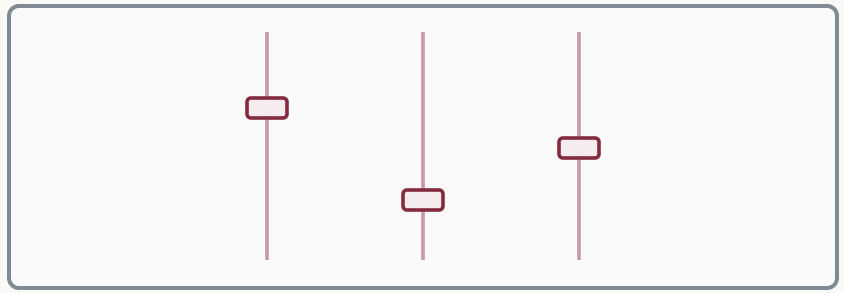
Can fewer trainable parameters lead to better accuracy?

Full fine-tuning



More freedom to change.
Can also fit noise in the data.

LoRA

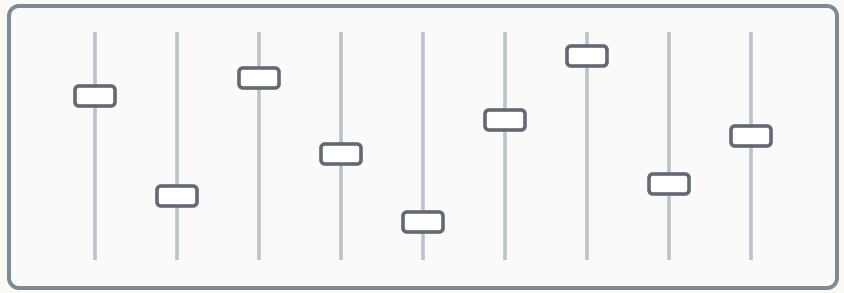


Fewer free parameters.
Can miss useful changes if too restricted.

LoRA vs. Full Fine-Tuning

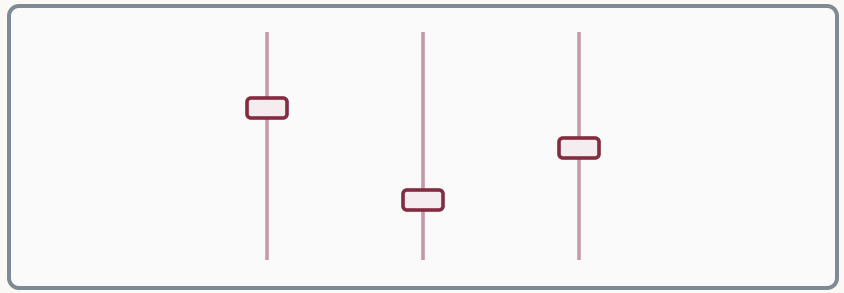
Can fewer trainable parameters lead to better accuracy?

Full fine-tuning



More freedom to change.
Can also fit noise in the data.

LoRA



Fewer free parameters.
Can miss useful changes if too restricted.

Main result

We show when LoRA can outperform full fine-tuning.

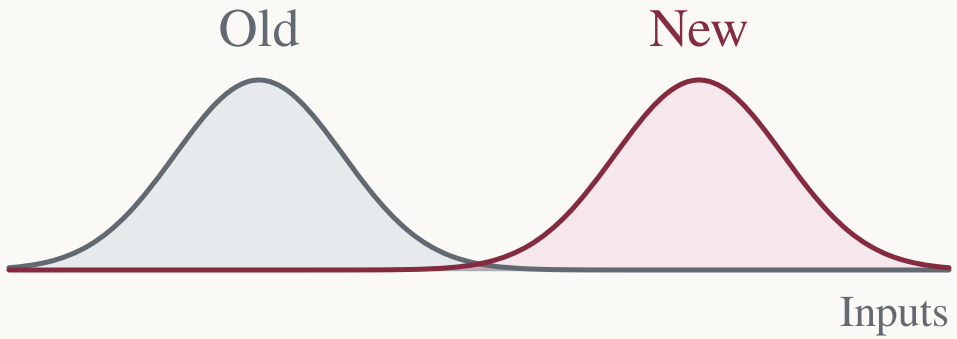
Theory of Adaptation and Retention

When can a model learn without forgetting?

Theory of Adaptation and Retention

When can a model learn without forgetting?

Little overlap

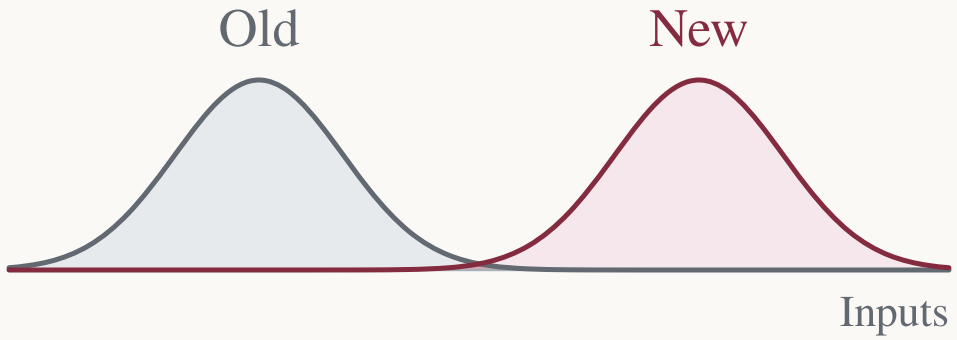


The model can learn the new task
and still do the old tasks well.

Theory of Adaptation and Retention

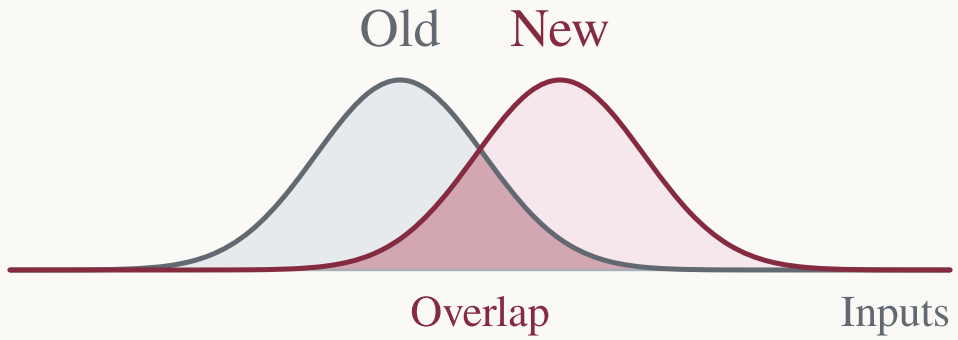
When can a model learn without forgetting?

Little overlap



The model can learn the new task
and still do the old tasks well.

More overlap

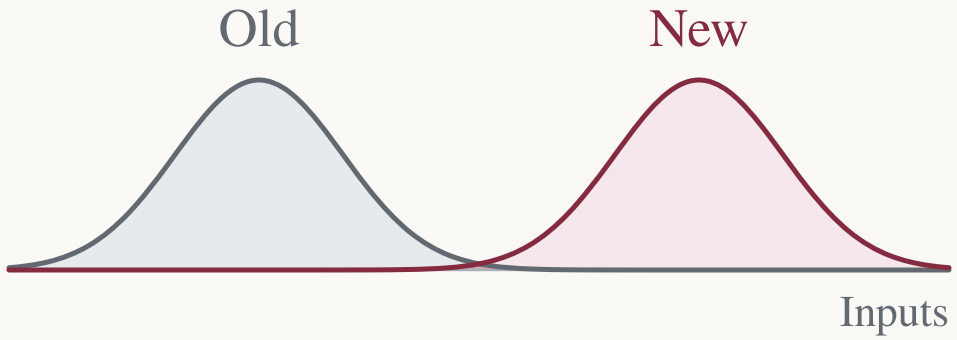


Same input, different required answers:
the model cannot get both tasks right.

Theory of Adaptation and Retention

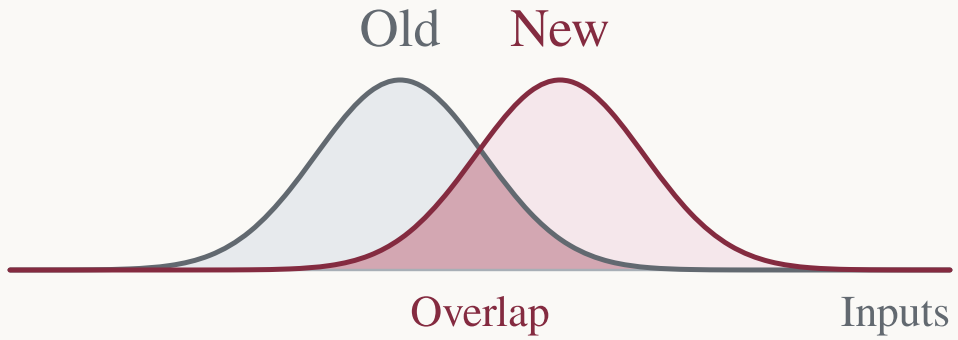
When can a model learn without forgetting?

Little overlap



The model can learn the new task
and still do the old tasks well.

More overlap



Same input, different required answers:
the model cannot get both tasks right.

Main result

We show when linear models can learn new tasks with little forgetting.

Future research directions

Models should keep learning and remembering important things.

Future research directions

Models should keep learning and remembering important things.



Adapt only where needed

Learn which parts of a model should change for each task.

Future research directions

Models should keep learning and remembering important things.



Adapt only where needed

Learn which parts of a model should change for each task.



Continual learning

Learn new tasks with little forgetting, even without old data.

Future research directions

Models should keep learning and remembering important things.



Adapt only where needed

Learn which parts of a model should change for each task.



Continual learning

Learn new tasks with little forgetting, even without old data.



Memory at test time

Remember useful information and recall it for later inputs.

Thank you

References

Methods and research

[1] Devlin et al. (2019). NAACL.
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

[2] Hu et al. (2022). ICLR.
LoRA: Low-Rank Adaptation of Large Language Models.

[3] Zindari et al. (2026). Under review.
Learning When to Adapt.

[4] Zindari et al. (2026). Under review.
LoRA vs. Full Fine-Tuning: A Theoretical Perspective.

[5] Zindari et al. (2026). Under review.
Theory of Adaptation and Retention.

Background

[6] Ouyang et al. (2022). NeurIPS.
Training Language Models to Follow Instructions with Human Feedback.

[7] Rafailov et al. (2023). NeurIPS.
Direct Preference Optimization: Your Language Model is Secretly a Reward Model.

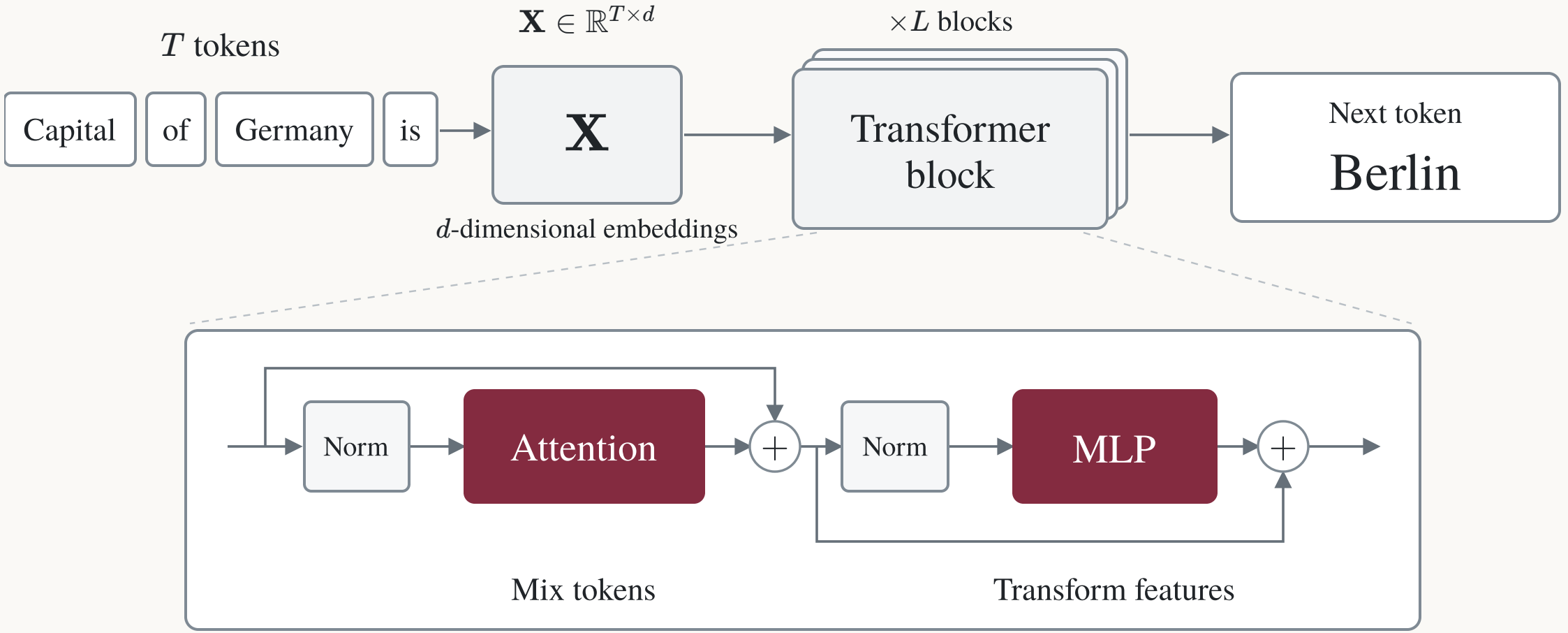
[8] Vaswani et al. (2017). NeurIPS.
Attention Is All You Need.

[9] Xiong et al. (2020). ICML.
On Layer Normalization in the Transformer Architecture.

[10] Vu et al. (2022). EMNLP.
Overcoming Catastrophic Forgetting in Zero-Shot Cross-Lingual Generation.

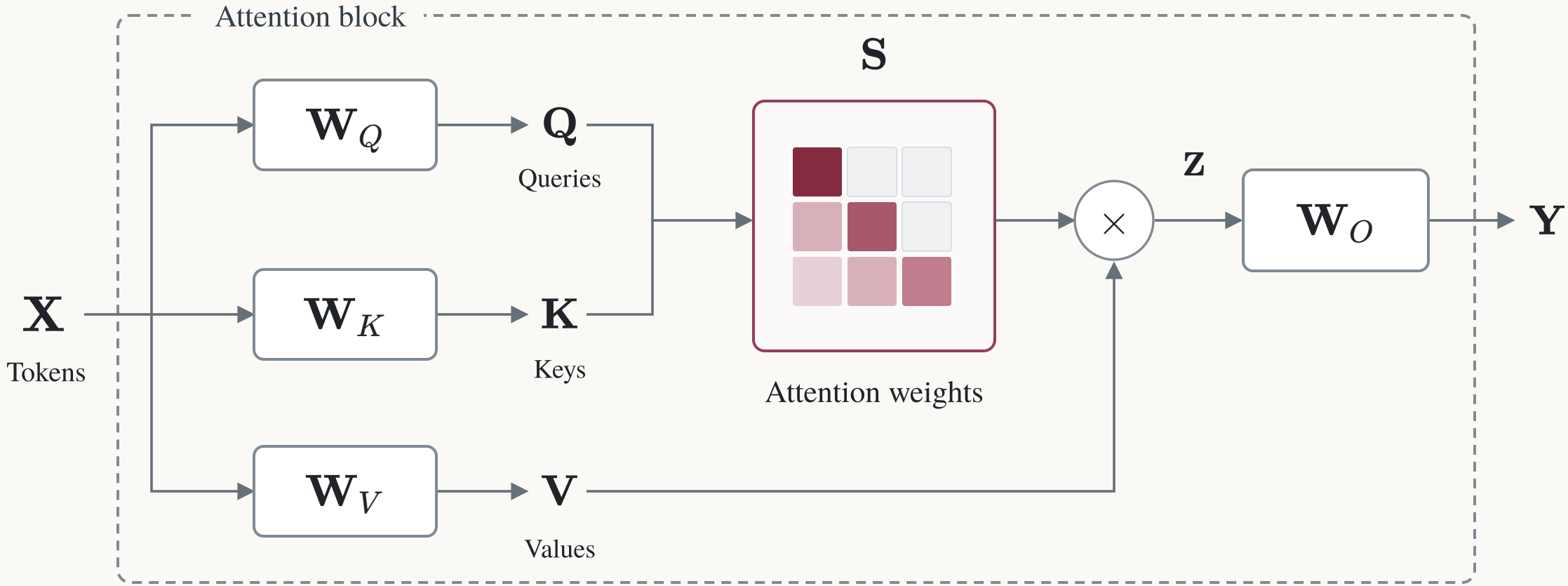
Appendix

A transformer updates token representations



Colored blocks contain weight matrices updated during full fine-tuning.

Inside attention: four learned projections



$$\mathbf{S} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right), \qquad \mathbf{Z} = \mathbf{S}\mathbf{V}$$

One attention head · $\mathbf{M}$ masks future tokens

DISeL: fine-tuning and task evaluation data

Two instruction-tuning settings

Setting	Fine-tuning data	Target-task evaluation
Math Llama 2-7B	MetaMathQA Math questions and worked answers	GSM8K Grade School Math · 5-shot exact match
Code Mistral-7B	Magicoder-Evol-Instruct-110K Code instructions and solutions	HumanEval Python programming problems · pass@1

Gate analysis: GSM8K (math), HumanEval (code), and WikiText-2 (general text).

Fine-tuning corpora: 95% training, 5% validation. The next slide lists all 14 retention benchmarks.

DISeL: all 14 LLM retention benchmarks

The same suite for Llama and Mistral · 5-shot · mean accuracy

HellaSwag
Commonsense sentence completion

WinoGrande
Winograd-style commonsense reasoning

ARC-Challenge
AI2 Reasoning Challenge · Challenge set

ARC-Easy
AI2 Reasoning Challenge · Easy set

LAMBADA — OpenAI version
Word prediction from a broad context

BoolQ
Natural yes/no questions

PIQA
Physical Interaction: Question Answering

TriviaQA
Trivia question answering

OpenBookQA
Open-book science question answering

SciQ
Science question answering

MMLU
Massive Multitask Language Understanding · 57 tasks

MedQA — 4 options
Medical question answering

CommonsenseQA
Commonsense question answering

Natural Questions — Open
Open-domain question answering

DISeL: RoBERTa fine-tuning and retention data

GLUE: General Language Understanding Evaluation

Fine-tuning and task evaluation

MNLI · Multi-Genre Natural Language Inference
MultiNLI · matched validation

SST-2 · Stanford Sentiment Treebank
Binary sentiment classification

QNLI · Question-answering Natural Language Inference
Derived from the Stanford Question Answering Dataset

CoLA · Corpus of Linguistic Acceptability
Grammatical acceptability

MRPC · Microsoft Research Paraphrase Corpus
Sentence-pair paraphrase classification

Retention evaluation

BookCorpusOpen
Open book-text corpus

CC-News · Common Crawl News
English news articles

WikiText-2
Wikipedia text · raw version

Mean masked-language-model perplexity on the three corpora.

Examples from retention datasets

ARC-Easy

Science knowledge

Which process best explains how the Grand Canyon became so wide?

A folding

B erosion

C deposition

D sedimentation

Correct answer: B · erosion

WinoGrande

Sentence understanding

Sarah was a much better surgeon than Maria so ____ always got the easier cases.

1 Sarah

2 Maria

Correct answer: 2 · Maria

Math fine-tuning data: a word problem

MetaMathQA · instruction and worked response

Instruction

Dave bought 8 books about animals, 6 books about outer space, and 3 books about trains to keep him busy over the holidays. Each book cost $6. How much did Dave spend on the books?

Response

Dave bought a total of $8 + 6 + 3 = 17$ books.

Each book cost $6, so Dave spent a total of $17 \times \$6 = \$102$ on the books.

The answer is: 102

Math fine-tuning data: a number-base problem

MetaMathQA · instruction and worked response

Instruction

Convert 10101_3 to a base 10 integer.

Response

$$10101_3 = 1 \cdot 3^4 + 0 \cdot 3^3 + 1 \cdot 3^2 + 0 \cdot 3^1 + 1 \cdot 3^0$$
$$= 81 + 9 + 1 = \boxed{91}.$$

The answer is: 91

Perplexity: scoring a test set

RoBERTa: score the masked tokens. Lower perplexity means better predictions.

Illustration: two test samples

Sample 1 · two masked tokens

The <mask> is <mask>.

sky: $p_1 = 0.5$ · blue: $p_2 = 0.25$

Sample 2 · one masked token

Birds can <mask>.

fly: $p_3 = 0.125$

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_i\right)$$

- p_i Probability of the correct token, given the visible text.
- N Total scored tokens across all test samples.

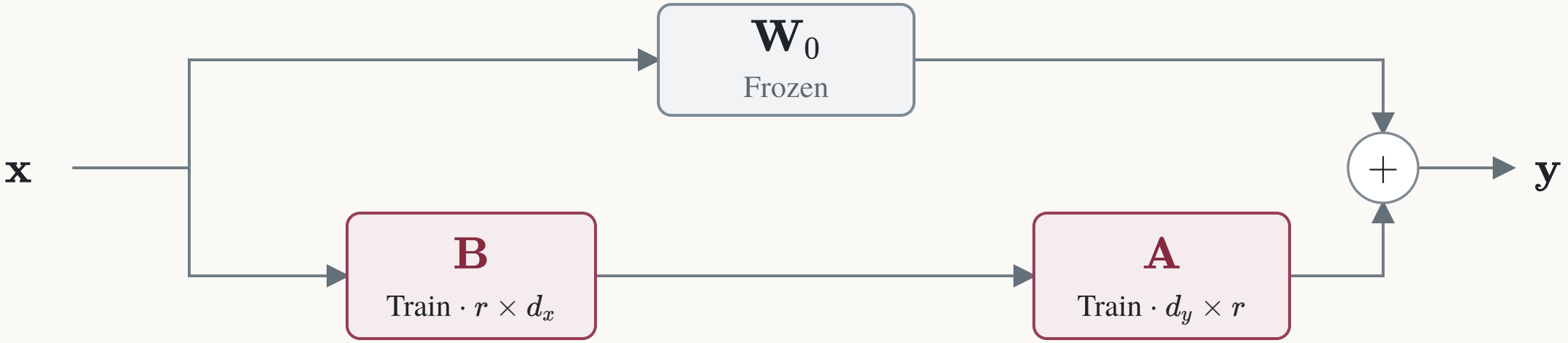
Here: $N = 2 + 1 = 3$, from two samples.

Sum the token losses $\rightarrow$ divide by N $\rightarrow$ exponentiate

$$\mathrm{PPL} = \exp\left(\frac{-\log 0.5 - \log 0.25 - \log 0.125}{3}\right) = 4$$

LoRA adds a low-rank path to the projection

$$\mathbf{y} = (\mathbf{W}_0 + \mathbf{AB})\mathbf{x} = \mathbf{W}_0\mathbf{x} + \mathbf{ABx}$$

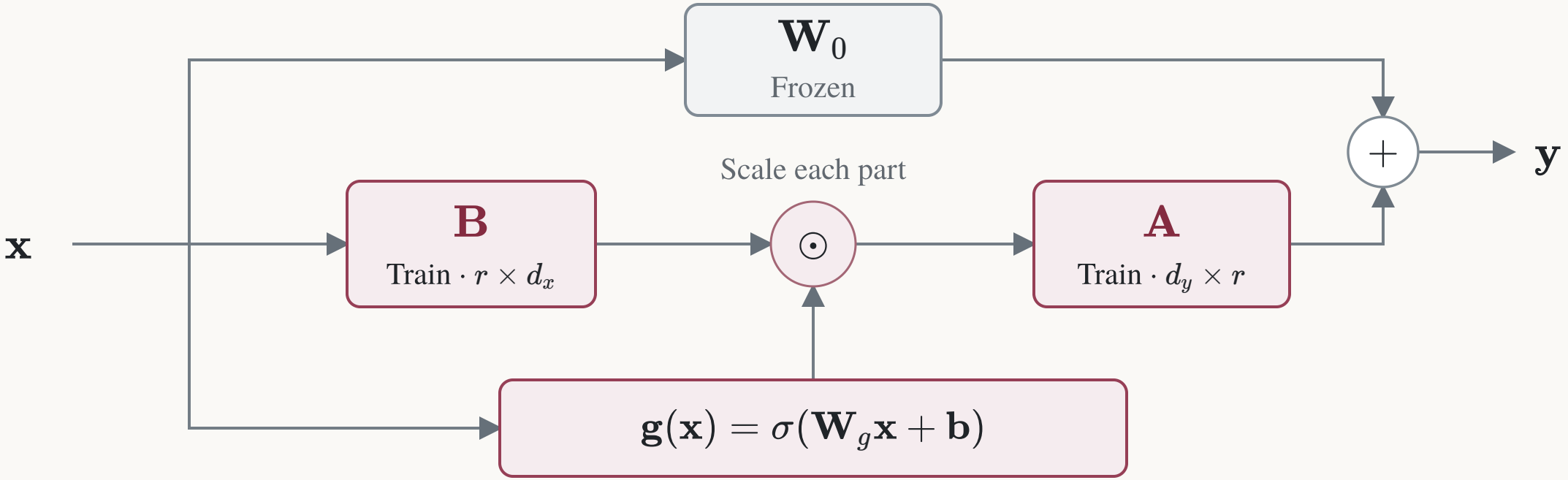


Gradients and optimizer states only for $\mathbf{A}$ and $\mathbf{B}$.

Trainable parameters	$r = 16$
$r(d_x + d_y)$, with $r \ll d_x, d_y$	131,072 128× fewer

The input controls the gates

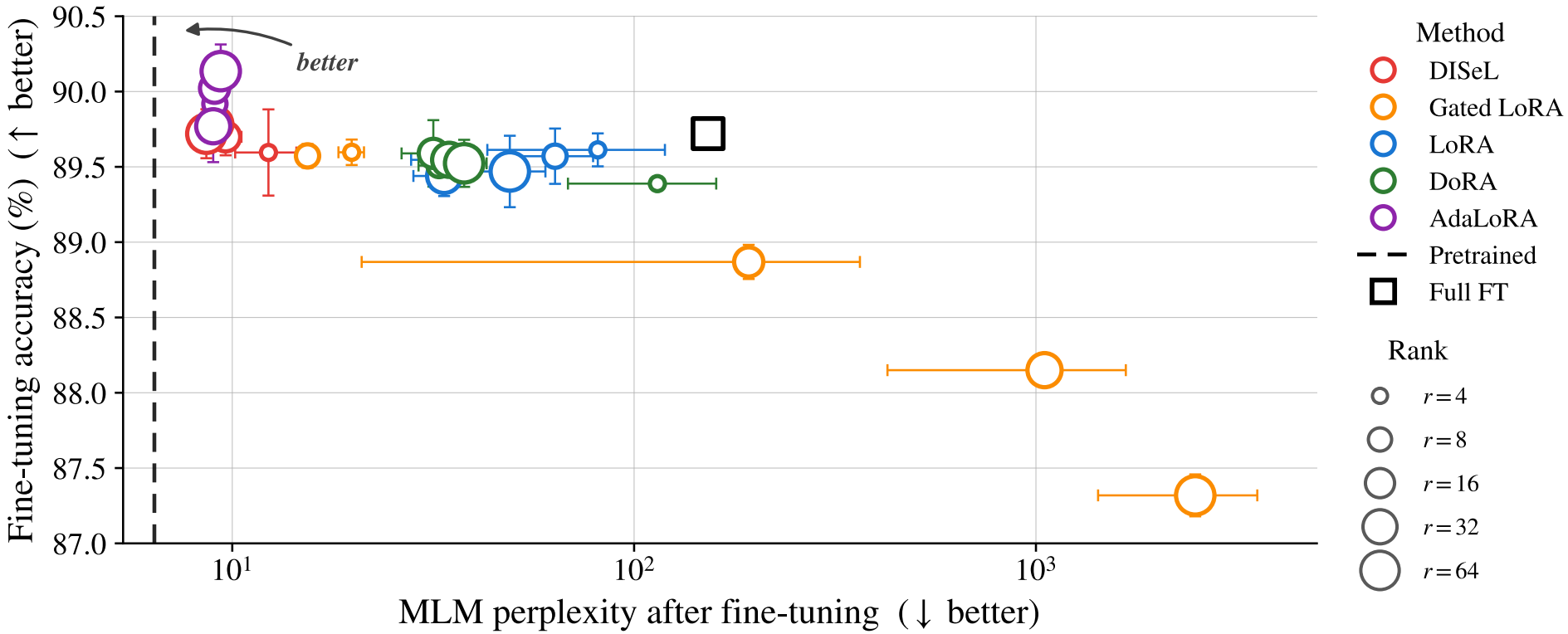
$$\mathbf{y} = \left(\mathbf{W}_0 + \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B}\right)\mathbf{x} = \mathbf{W}_0\mathbf{x} + \mathbf{A}\mathbf{G}(\mathbf{x})\mathbf{B}\mathbf{x}$$



We train the update and gates together on new-task data.

Trainable: $r(d_y + 2d_x + 1)$ $\quad d_x = d_y = 4096,\ r = 16:$ $\quad$ 196,624 ($\approx 1.5\times$ LoRA)

Strong task accuracy can hide severe forgetting



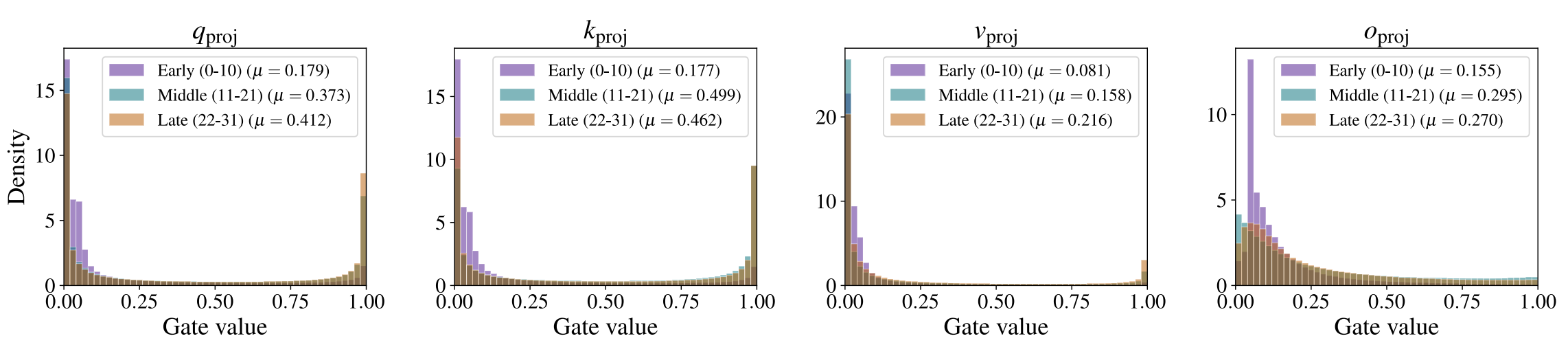
Here, **upper left** is better. Points average three seeds; bars show standard deviations.

Comparison across scales

DISeL and AdaLoRA are similar here, but DISeL's advantage appears at the larger 7B scale.

Gate activation across attention matrices

Llama 2-7B · Rank-32 math adapter · Held-out GSM8K



Query and key gates are more open on average than value gates.

[3] Zindari et al. (2026). *Learning When to Adapt.* Fig. 10, attention panels.

DISeL: overhead and limitations

Training overhead

More parameters than LoRA:
$+\,r(d_x + 1)$ per matrix.
Extra training memory.

Inference cost

Gates run for every token.
The update cannot be merged
into fixed weights.

Retention limits

Some skills may still be lost.
Depends on initialization
and training.

[3] Zindari et al. (2026). *Learning When to Adapt.* §3 and §8.